

# AUTOMATED DETECTION, TRACKING AND TACTICAL ANALYSIS IN BROADCAST FOOTBALL VIDEO

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER  
FOR THE DEGREE OF BACHELOR OF SCIENCE  
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

**Aryan Tiwari**

Supervised by Dr Aphrodite Galata

Department of Computer Science

# Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                  | <b>10</b> |
| <b>Declaration</b>                                               | <b>11</b> |
| <b>Copyright</b>                                                 | <b>12</b> |
| <b>Acknowledgements</b>                                          | <b>13</b> |
| <b>1 Introduction</b>                                            | <b>14</b> |
| 1.1 Motivation and Context . . . . .                             | 14        |
| 1.2 Aims and Objectives . . . . .                                | 15        |
| 1.3 Evaluation Strategy . . . . .                                | 16        |
| 1.4 Report Structure . . . . .                                   | 17        |
| <b>2 Background and Theory</b>                                   | <b>19</b> |
| 2.1 Broadcast Football Video Analysis . . . . .                  | 19        |
| 2.2 Related Work . . . . .                                       | 20        |
| 2.2.1 Commercial Tracking Systems . . . . .                      | 20        |
| 2.2.2 Academic Benchmarks and Integrated Pipelines . . . . .     | 20        |
| 2.2.3 Positioning of This Project . . . . .                      | 21        |
| 2.3 Machine Learning and Computer Vision Foundations . . . . .   | 21        |
| 2.3.1 Supervised Learning for Visual Recognition . . . . .       | 22        |
| 2.3.2 Neural Networks . . . . .                                  | 22        |
| 2.3.3 Convolutional Neural Networks . . . . .                    | 23        |
| 2.3.4 Transfer Learning and Fine-Tuning . . . . .                | 24        |
| 2.4 Object Detection in Broadcast Football . . . . .             | 24        |
| 2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression . . . . . | 25        |
| 2.4.2 Precision, Recall, and Mean Average Precision . . . . .    | 26        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 2.4.3    | The YOLO Architecture . . . . .                        | 27        |
| 2.5      | Multi-Object Tracking . . . . .                        | 29        |
| 2.5.1    | Tracking-by-Detection . . . . .                        | 30        |
| 2.5.2    | Motion Prediction and Data Association . . . . .       | 30        |
| 2.5.3    | ByteTrack . . . . .                                    | 31        |
| 2.5.4    | BoT-SORT . . . . .                                     | 33        |
| 2.5.5    | Tracking Evaluation: HOTA . . . . .                    | 33        |
| 2.6      | Team Identification from Visual Features . . . . .     | 33        |
| 2.6.1    | Team Identification as Clustering . . . . .            | 34        |
| 2.6.2    | Visual Representations . . . . .                       | 34        |
| 2.6.3    | Dimensionality Reduction . . . . .                     | 35        |
| 2.6.4    | $k$ -Means Clustering . . . . .                        | 36        |
| 2.7      | Pitch Registration and Planar Homography . . . . .     | 36        |
| 2.7.1    | Planar Homography . . . . .                            | 37        |
| 2.7.2    | Estimating $H$ from Point Correspondences . . . . .    | 37        |
| 2.7.3    | Pitch Keypoint Detection . . . . .                     | 38        |
| 2.7.4    | Reprojection Error . . . . .                           | 39        |
| 2.8      | Spatial and Tactical Analysis in Pitch Space . . . . . | 39        |
| 2.8.1    | Pitch-Space Trajectories . . . . .                     | 39        |
| 2.8.2    | Heatmap Construction . . . . .                         | 40        |
| 2.8.3    | Speed and Distance Estimation . . . . .                | 40        |
| 2.8.4    | Possession via Nearest-Player Assignment . . . . .     | 41        |
| 2.8.5    | Offside Rule Formalisation . . . . .                   | 42        |
| 2.9      | Chapter Summary . . . . .                              | 42        |
| <b>3</b> | <b>Methodology and Implementation</b>                  | <b>44</b> |
| 3.1      | Chapter Overview . . . . .                             | 44        |
| 3.2      | Requirements and Design Objectives . . . . .           | 45        |
| 3.2.1    | Functional Requirements . . . . .                      | 45        |
| 3.2.2    | Non-functional Requirements . . . . .                  | 46        |
| 3.2.3    | Success Criteria . . . . .                             | 48        |
| 3.3      | System Architecture . . . . .                          | 49        |
| 3.3.1    | Pipeline Overview . . . . .                            | 49        |
| 3.3.2    | Data Flow and Intermediate Representations . . . . .   | 51        |
| 3.3.3    | System Outputs . . . . .                               | 54        |
| 3.4      | Datasets and Model Training . . . . .                  | 55        |

|        |                                                               |    |
|--------|---------------------------------------------------------------|----|
| 3.4.1  | Datasets . . . . .                                            | 55 |
| 3.4.2  | Generic Object Detector Training . . . . .                    | 56 |
| 3.5    | Small-Object Detection and Dedicated Ball Detection . . . . . | 57 |
| 3.5.1  | Why Small Objects Are Harder to Detect . . . . .              | 57 |
| 3.5.2  | Strategies for Small-Object Detection . . . . .               | 58 |
| 3.5.3  | Temporal Filtering of Ball Candidates . . . . .               | 59 |
| 3.5.4  | Dedicated Ball Detector Training . . . . .                    | 59 |
| 3.5.5  | Pitch Keypoint Detector Training . . . . .                    | 60 |
| 3.6    | Detection and Tracking . . . . .                              | 61 |
| 3.6.1  | Main Object Detection . . . . .                               | 61 |
| 3.6.2  | Dedicated Ball Detection and Filtering . . . . .              | 61 |
| 3.6.3  | Multi-object Tracking . . . . .                               | 62 |
| 3.6.4  | Track Handling and Failure Cases . . . . .                    | 63 |
| 3.7    | Team Identification . . . . .                                 | 63 |
| 3.7.1  | Warm-up Sampling and Player Cropping . . . . .                | 63 |
| 3.7.2  | Embedding and Clustering . . . . .                            | 64 |
| 3.7.3  | Temporal Smoothing and Role Handling . . . . .                | 64 |
| 3.8    | Pitch Registration . . . . .                                  | 65 |
| 3.8.1  | Pitch Keypoint Detection . . . . .                            | 65 |
| 3.8.2  | Homography Estimation and Coordinate Transformation . . . . . | 65 |
| 3.8.3  | Caching and Robustness Measures . . . . .                     | 66 |
| 3.9    | Tactical Output Generation . . . . .                          | 66 |
| 3.9.1  | Bird’s-eye Visualisation . . . . .                            | 67 |
| 3.9.2  | Heatmaps, Speed and Distance . . . . .                        | 67 |
| 3.9.3  | Possession and Pass Detection . . . . .                       | 68 |
| 3.9.4  | Offside Candidate Detection . . . . .                         | 68 |
| 3.10   | Evaluation Methodology . . . . .                              | 69 |
| 3.10.1 | Component-level Evaluation . . . . .                          | 69 |
| 3.10.2 | End-to-end Pipeline Evaluation . . . . .                      | 70 |
| 3.10.3 | Ablation Studies . . . . .                                    | 70 |
| 3.10.4 | Runtime Evaluation . . . . .                                  | 71 |
| 3.10.5 | Error Analysis . . . . .                                      | 71 |
| 3.11   | Implementation and Reproducibility . . . . .                  | 72 |
| 3.11.1 | Software Environment . . . . .                                | 72 |
| 3.11.2 | Configuration and Model Weights . . . . .                     | 73 |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 3.11.3   | Running the Pipeline . . . . .        | 73        |
| 3.11.4   | Reproducibility Limitations . . . . . | 74        |
| 3.12     | Chapter Summary . . . . .             | 74        |
| <b>4</b> | <b>Results</b>                        | <b>76</b> |
| 4.1      | Experimental Setup . . . . .          | 76        |
| 4.2      | Findings . . . . .                    | 76        |
| <b>5</b> | <b>Evaluation</b>                     | <b>77</b> |
| 5.1      | Analysis . . . . .                    | 77        |
| 5.2      | Discussion . . . . .                  | 77        |
| 5.3      | Limitations . . . . .                 | 77        |
| <b>6</b> | <b>Conclusion</b>                     | <b>78</b> |
| 6.1      | Summary . . . . .                     | 78        |
| 6.2      | Future Work . . . . .                 | 78        |
| <b>A</b> | <b>Additional Materials</b>           | <b>84</b> |

**Word Count: 14700 (TODO: update at the end)**

# List of Tables

|     |                                                                                                                                                                                                                                                   |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Summary of evaluation metrics used in this project. . . . .                                                                                                                                                                                       | 17 |
| 3.1 | Data objects passed between pipeline stages. The <code>supervision.Detections</code> type is used throughout the left branch of the pipeline with fields progressively populated and, in the case of <code>class_id</code> , reinterpreted. . . . | 52 |
| 3.2 | Summary of the three training datasets. All datasets are hosted on Roboflow Universe and were downloaded via the Roboflow Python SDK; splits are those provided by the dataset version on Roboflow. . .                                           | 55 |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | A common CNN architecture: stacked convolution, activation, and pooling stages produce feature maps of decreasing spatial resolution, which are passed to fully connected layers for prediction. Reproduced from O'Shea and Nash [34]. . . . .                                                                                                                                                                                                                                                                                                   | 24 |
| 2.2 | Intersection over Union (IoU) between a ground-truth bounding box $B_g$ (blue, dashed) and a predicted bounding box $B_p$ (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} =  B_g \cap B_p  /  B_g \cup B_p $ . . . . .                                                                                                                                                                                                                                                 | 25 |
| 2.3 | Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained. . . . .                                                                                                                                                                                                                                                                                                | 26 |
| 2.4 | Generalised YOLO-style single-stage detection pipeline. The backbone extracts multi-scale feature maps, the neck fuses them, and the detection head predicts boxes, objectness, and class scores. NMS removes duplicate predictions to produce the final detection set $\{(b_i, c_i, s_i)\}$ . . . . .                                                                                                                                                                                                                                           | 28 |
| 2.5 | ByteTrack two-stage association process. Detector output is partitioned by confidence threshold $\tau_{\text{high}}$ into a high-score set $D_t^{\text{high}}$ and a low-score set $D_t^{\text{low}}$ . Stage 1 matches active tracks against $D_t^{\text{high}}$ using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against $D_t^{\text{low}}$ . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames. . . . . | 32 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | End-to-end pipeline architecture. The broadcast frame enters at the top and is processed through two parallel branches—appearance-based detection and tracking on the left, pitch-geometry recovery on the right—which converge at the pitch-space projection stage. All higher-level tactical analytics operate in pitch coordinates. Section numbers on the left indicate where each stage is described in detail. . . . .                                                                | 50 |
| 3.2 | Effect of pixel displacement $\delta$ on IoU for square bounding boxes of varying size. A single-axis shift is applied to the predicted box while the ground-truth box remains fixed. At $\delta = 2$ pixels, a $16 \times 16$ box drops to $\text{IoU} = 0.78$ while a $64 \times 64$ box retains $\text{IoU} = 0.94$ , illustrating why small-object detection is disproportionately sensitive to localisation error. The dashed line marks the standard $\text{mAP}_{50}$ threshold. . . | 58 |



# List of Abbreviations

|      |                                |
|------|--------------------------------|
| HOTA | Higher Order Tracking Accuracy |
| IDF1 | Identification F1              |
| IoU  | Intersection over Union        |
| MAE  | Mean Absolute Error            |
| mAP  | mean Average Precision         |
| MOTA | Multi-Object Tracking Accuracy |

# Abstract

TODO: my abstract (to be done at the end of the report)

# **Declaration**

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

# Copyright

- i. The author of this report (including any appendices and/or schedules to this report) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this report, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the report, for example graphs and tables (“Reproductions”), which may be described in this report, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this report, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

# Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Aphrodite Galata, for her invaluable guidance, support, and advice throughout this project.

I am also grateful to Kunj Dhola and Selim Sayed for their collaboration and contributions to the project.

Finally, I would like to thank my family and friends for their continued encouragement and support throughout my studies.

# Chapter 1

## Introduction

### 1.1 Motivation and Context

The use of data and analytics in professional football has grown substantially in recent years, with clubs and national federations increasingly incorporating analytical methods into performance analysis and decision-making [27]. In particular, positional and tracking data are valuable because they provide richer descriptions of player movement, team organisation, and tactical behaviour than traditional notational analysis alone [11, 43]. As a result, access to reliable spatial match data has become increasingly important.

Despite this, high-quality football data remain difficult to access outside elite settings. Advanced event and tracking data are often proprietary, expensive, or dependent on specialist collection systems, which limits their availability for public and reproducible work [24, 9, 6]. This motivates the investigation of alternative ways to generate structured football data from more widely available sources.

Broadcast video is an attractive candidate because it is widely available and already contains much of the spatial information needed for tactical analysis. However, extracting reliable structured data from broadcast footage is challenging, since it requires several interconnected tasks such as player detection, tracking, team assignment, and pitch registration under conditions including camera motion, occlusion, scale variation, and motion blur [25, 3, 42, 41].

Recent research has shown progress on many of these components and, increasingly, on parts of the broader workflow. Open benchmark efforts such as SoccerNet-Tracking and SoccerNet Game State Reconstruction demonstrate that broadcast football video can support tracking and game-state reconstruction in realistic settings [5,

39]. However, these systems typically produce intermediate outputs such as tracked player positions or minimap reconstructions rather than directly usable tactical outputs. What remains missing is a publicly reproducible end-to-end system that integrates all major stages from broadcast video to tactical output using modern computer vision methods.

## 1.2 Aims and Objectives

To address this gap, the project investigates how far a modular computer vision pipeline can extract reliable tactical and spatial information from broadcast football video without requiring specialised capture systems or manual team labels.

To achieve this aim, the project pursues the following objectives:

- develop and evaluate a detection approach for the main on-pitch entity classes—players, goalkeepers, referees, and the ball—from broadcast football footage;
- assess whether multi-object tracking can preserve sufficiently consistent identities over time to support downstream tactical analysis;
- test whether unsupervised clustering of visual embeddings can separate players by team without manual team labels;
- measure the geometric accuracy of pitch keypoint detection and homography estimation for projecting image-space positions into pitch-space;
- determine whether the resulting projected trajectories can support higher-level tactical outputs, including bird’s-eye visualisations, heatmaps, possession estimates, pass events, and offside-related analysis; and
- evaluate the pipeline as an end-to-end system, including how errors in detection, tracking, and homography estimation affect downstream analytical outputs.

The project does not aim to replace professional commercial tracking systems, but rather to assess how much tactical information a reproducible, open pipeline can extract from standard broadcast footage.

The project will be considered successful if: (i) the main object detector achieves  $\text{mAP}_{50} \geq 0.80$  across entity classes; (ii) multi-object tracking reaches  $\text{HOTA} \geq 0.45$  on a labelled test segment; (iii) unsupervised team assignment achieves  $F_1 \geq 0.85$ ; and

(iv) the integrated pipeline produces coherent tactical outputs—including bird’s-eye projections, heatmaps, and offside analysis—that are qualitatively consistent with the underlying match footage. These thresholds reflect realistic performance expectations for broadcast football analysis informed by the literature [42, 5].

### 1.3 Evaluation Strategy

This project will be evaluated at both *component level* and *system level*. This is necessary because downstream outputs such as possession estimates, heatmaps, and offside decisions depend not only on the performance of individual modules, but also on how errors propagate across detection, tracking, and pitch registration stages [42, 41]. The evaluation therefore asks two questions: whether each component performs adequately on its own task, and whether the integrated pipeline produces coherent and tactically meaningful outputs as a complete system.

At component level, each module will be assessed using standard task-appropriate metrics from the computer vision literature. At system level, the pipeline will be judged by the coherence, plausibility, and usefulness of its final outputs on representative broadcast sequences. The chosen metrics reflect the main technical risks of the project: detection accuracy, temporal identity consistency, geometric projection quality, and the validity of downstream tactical inference. These metrics were chosen because each targets a distinct failure mode of the pipeline. Mean Average Precision (mAP) is the standard object detection metric in the computer vision literature and directly measures whether the detector can reliably localise and classify entities under broadcast conditions [42]. Higher Order Tracking Accuracy (HOTA) was selected over legacy Multi-Object Tracking (MOT) metrics such as Multi-Object Tracking Accuracy (MOTA) because it balances detection and association quality in a single score, making it better suited to evaluating identity consistency over time [28]. Reprojection error measures how accurately estimated homographies map known pitch points, which determines whether projected player positions are geometrically meaningful.

At system level, quantitative metrics alone cannot capture whether tactical outputs are useful, so qualitative assessment against the source footage is included following the approach used in recent game-state reconstruction benchmarks [39]. The numeric thresholds were set based on reported performance in comparable broadcast football settings: for example, Theiner et al. [42] report mAP<sub>50</sub> values in the range 0.80–0.90 for player detection, and SoccerNet-Tracking baselines report HOTA scores



in the 0.40–0.55 range [5], making the chosen targets realistic but non-trivial. Detailed annotation procedures and experimental methodology are presented in Chapter 5.

Table 1.1: Summary of evaluation metrics used in this project.

| Component / Output             | Primary metrics                                                                                                                | Purpose                                                                                               |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Object detection               | mAP <sub>50–95</sub> , mAP <sub>50</sub> , precision, recall                                                                   | Measure localisation and classification performance for players, goalkeepers, referees, and the ball. |
| Ball detection                 | mAP <sub>50</sub> , precision, recall, false positive rate                                                                     | Assess robustness under small object size, motion blur, and occlusion.                                |
| Pitch keypoints and homography | mAP <sub>50–95</sub> for keypoints; reprojection error for homography                                                          | Measure field registration quality and projection accuracy in pitch-space.                            |
| Multi-object tracking          | HOTA, IDF1, MOTA, identity switches                                                                                            | Evaluate temporal consistency and identity preservation [28, 8].                                      |
| Team assignment                | Precision, recall, $F_1$ score                                                                                                 | Measure accuracy of unsupervised team separation.                                                     |
| Tactical outputs               | Precision, recall, and $F_1$ for event detection; MAE for possession; qualitative assessment for heatmaps and bird’s-eye views | Assess whether projected trajectories support useful downstream analysis.                             |
| End-to-end pipeline            | Integrated case study and failure analysis                                                                                     | Evaluate output coherence and identify how upstream errors affect downstream reasoning.               |

Component-level evaluation shows whether each stage meets its technical objective, while system-level evaluation tests whether the combined pipeline remains analytically useful once these stages are linked. The final judgement of success will therefore be based on both quantitative component metrics and qualitative assessment of whether the integrated outputs remain consistent with the source footage and useful for tactical interpretation.

## 1.4 Report Structure

The remainder of this report is organised to guide the reader from the background knowledge needed to understand the system, through its design and outputs, to a critical assessment of its performance. Chapter 2 (*Background and Theory*) reviews the relevant literature and technical foundations, establishing the context from which the pipeline design decisions follow. Chapter 3 (*Methodology and Implementation*) describes the architecture and implementation of each pipeline stage, building directly on

the methods and models introduced in the background. Chapter 4 (*Results*) presents the outputs produced by the system in practice. Chapter 5 (*Evaluation*) then assesses the project at both component and system level, using the metrics and strategy outlined in Section 1.3, including failure analysis and the effect of error propagation on downstream outputs. Finally, Chapter 6 (*Conclusion*) summarises the main achievements, reflects on limitations, and outlines directions for future work.

# Chapter 2

## Background and Theory

### 2.1 Broadcast Football Video Analysis

Broadcast football footage poses several coupled technical challenges that distinguish it from analysis based on dedicated tracking infrastructure [42, 25]. Where multi-camera or sensor-based systems can rely on fixed reference frames and direct measurement, a broadcast pipeline must recover all player, ball, and pitch information from a single moving-camera video stream [11].

Four properties of this medium drive the design of any such pipeline. The camera continuously pans, tilts, and zooms, so the background cannot serve as a fixed spatial reference and inter-frame motion cannot be attributed to scene objects alone [42]. The ball is small relative to the frame and is frequently blurred, occluded, or temporarily absent, making its detection substantially harder than that of players [5]. Players on the same team share near-identical appearance — uniform colour, similar build — so identity must be inferred from subtle spatial and temporal cues [42]. Finally, image-space measurements suffer perspective distortion, so quantities such as distance, speed, and tactical position are meaningful only after registration to a canonical pitch model [41].

These constraints have driven the field toward a modular pipeline [42, 25]: object and ball detection produce per-frame observations; multi-object tracking maintains temporal identity; team assignment recovers affiliation; pitch registration projects coordinates into field space; and higher-level outputs — heatmaps, possession, offside — are computed in pitch coordinates rather than in the image plane.

Formally, the pipeline is a composition of stages,

$$y = f_n(f_{n-1}(\cdots f_1(x))), \quad (2.1)$$

where  $x$  is the input video, each  $f_k$  is a processing stage, and  $y$  is a final pitch-space output such as a heatmap, possession estimate, or offside decision. This composed structure has an important consequence: every output depends on the quality of every upstream stage, so the pipeline cannot be analysed by considering its components in isolation.

## 2.2 Related Work

This section reviews existing commercial systems, public benchmarks, and academic pipelines for broadcast football analysis, in order to identify the gap that motivates an open pipeline producing tactical outputs from broadcast video alone.

### 2.2.1 Commercial Tracking Systems

Professional clubs typically obtain positional data from commercial optical or sensor-based systems. TRACAB uses multi-camera optical tracking and has been shown to attain sub-metre positional accuracy under validated match conditions [24], while providers such as Hudl StatsBomb and Second Spectrum supply event data, tracking data, and tactical analysis tools to clubs and broadcasters [6]. These systems are not, however, a viable foundation for open research: their algorithms, data-collection procedures, and raw outputs are proprietary, which limits reproducibility, fair comparison across methods, and applicability to matches where commercial tracking data are unavailable [9, 6]. This motivates approaches that recover spatial and tactical information directly from broadcast footage.

### 2.2.2 Academic Benchmarks and Integrated Pipelines

The SoccerNet ecosystem provides the largest open benchmark suite for broadcast football. SoccerNet-v2 introduced annotations for action spotting, camera-shot segmentation, and replay detection [7]; SoccerNet-Tracking added bounding-box annotations and standard evaluation protocols for multi-object tracking [5]; and SoccerNet Game State Reconstruction frames the problem as recovering the full game state on

a top-down minimap, jointly evaluating detection, tracking, identification, and pitch registration [39].

Integrated academic systems address overlapping subsets of this pipeline. Theiner et al. [42] extract positional data by combining detection, tracking, and pitch registration, but evaluate only positional accuracy rather than downstream tactical reasoning. Magera et al. [30] (BroadTrack) target broadcast camera calibration specifically, leaving downstream stages out of scope. Maglo et al. [31] pair field registration with self-supervised re-identification to localise players from a single moving view. Earlier work by Beetz et al. [3] demonstrated the feasibility of automated game analysis but predates modern deep-learning detectors and trackers.

A consistent pattern is that these systems stop at intermediate representations — bounding boxes, tracks, registered pitches, or minimap reconstructions. A reconstruction may be visually plausible while still containing localisation, identity, or ball-detection errors that propagate into derived quantities such as possession, pass detection, distance estimates, or offside decisions. Evaluation at the reconstruction level alone therefore does not establish tactical usefulness.

### **2.2.3 Positioning of This Project**

The literature thus leaves a clear gap: closed commercial systems produce tactical outputs but are not reproducible, while open academic systems are reproducible but stop short of tactical interpretation. This project addresses that gap by extending an open broadcast pipeline beyond reconstruction, treating detection, tracking, and pitch registration as inputs to downstream pitch-space statistics rather than as final outputs.

## **2.3 Machine Learning and Computer Vision Foundations**

This section introduces the machine learning and computer vision concepts on which the project depends, so that the project-specific methods in later sections are accessible to a reader without prior familiarity with these techniques.

### 2.3.1 Supervised Learning for Visual Recognition

In supervised learning, a model learns from labelled examples. A training set

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

pairs each input  $x_i$  — for instance an image or image crop — with a label  $y_i$ , which may be an object class, a bounding box, a set of keypoints, or a combination of these. The model is a function  $f_\theta$  with learnable parameters  $\theta$ , and training minimises the empirical risk

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i),$$

where  $\mathcal{L}$  measures the discrepancy between prediction and ground truth. Two losses recur in this project. Cross-entropy is used for classification targets,

$$\mathcal{L}_{\text{CE}}(\hat{y}, y) = - \sum_{c=1}^C y_c \log \hat{y}_c,$$

where  $\hat{y}$  is a predicted distribution over  $C$  classes and  $y$  is a one-hot ground truth. Squared error is used for continuous targets such as bounding-box coordinates,

$$\mathcal{L}_{\text{MSE}}(\hat{y}, y) = \|\hat{y} - y\|_2^2.$$

Optimisation is performed by stochastic gradient descent or one of its variants, with parameters updated as

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L},$$

where  $\eta$  is the learning rate [12]. The supervised components of this project — the object detector and the pitch keypoint detector — all instantiate this framework, learning visual patterns from annotated examples rather than from hand-written rules.

### 2.3.2 Neural Networks

A neural network is a parametric model composed of layers of simple computational units. Each unit combines its inputs through learned weights and biases and applies a non-linear activation. For layer  $l$ ,

$$\mathbf{h}^{(l)} = \sigma\left(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right),$$

where  $W^{(l)}$  and  $\mathbf{b}^{(l)}$  are the layer’s learnable parameters and  $\sigma$  is an element-wise non-linearity. The non-linearity is essential: a composition of linear maps is itself linear, so without  $\sigma$  the entire network would collapse to a single linear transformation. The most common choice is the rectified linear unit,

$$\text{ReLU}(z) = \max(0, z),$$

which is computationally cheap and tends to produce sparse activations that aid optimisation [12]. A fully-connected network of this form, however, treats an image as a flat vector — which scales poorly with image size and discards spatial structure. Convolutional architectures, introduced next, address both issues.

### 2.3.3 Convolutional Neural Networks

Convolutional neural networks (CNNs) are designed for grid-structured data such as images. Rather than connecting every input to every output, a CNN applies small learned filters across local regions of the image, so the same visual pattern — an edge, a texture, a goalpost — can be detected regardless of its position in the frame. A two-dimensional convolutional layer computes

$$y_{i,j,k} = \sigma \left( \sum_{u,v,c} W_{u,v,c,k} x_{i+u,j+v,c} + b_k \right),$$

where  $x$  is the input feature map,  $W$  is a learned filter,  $b_k$  is a bias, and  $y_{i,j,k}$  is the output at spatial location  $(i, j)$  on output channel  $k$ ; the indices  $u, v$  range over the filter’s spatial extent and  $c$  over input channels. Weight sharing across spatial positions makes CNNs both more parameter-efficient than fully-connected networks and translation-equivariant by construction.

Pooling layers reduce spatial resolution and enlarge the effective receptive field of subsequent filters. Max pooling, the most common variant, takes

$$y_{i,j} = \max_{(u,v) \in \mathcal{R}_{i,j}} x_{u,v}$$

over a local region  $\mathcal{R}_{i,j}$ . Stacking convolution, activation, and pooling yields a hierarchical representation: early layers respond to local patterns such as edges and colour gradients, while deeper layers compose these into semantic features such as body parts, kit textures, or pitch markings. This compositional behaviour was already

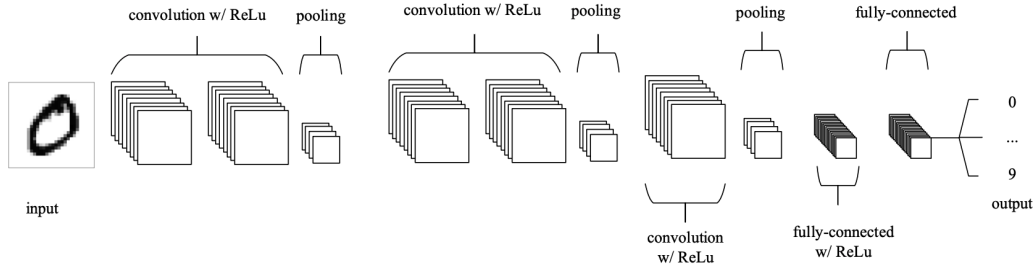


Figure 2.1: A common CNN architecture: stacked convolution, activation, and pooling stages produce feature maps of decreasing spatial resolution, which are passed to fully connected layers for prediction. Reproduced from O’Shea and Nash [34].

present in early systems such as LeNet [21], and deep CNNs became dominant in computer vision after their breakthrough on large-scale image classification [19]. In this project, CNN backbones provide the visual features for both object detection and pitch keypoint localisation; the detection-specific architecture of YOLO is discussed in Section 2.4.

### 2.3.4 Transfer Learning and Fine-Tuning

Training deep visual models from scratch typically requires very large datasets and substantial computation. Transfer learning sidesteps this by initialising the target-task model with parameters  $\theta_0$  learned from a large general-purpose dataset such as ImageNet or COCO. Empirically, the lower layers of such pre-trained networks encode general visual features — edges, colour transitions, textures — that transfer well across domains, while upper layers are more task-specific [44]. Fine-tuning continues training from  $\theta_0$  on the target task, often with a smaller learning rate so that useful low-level features are preserved while higher-level representations adapt. This regime is well-suited to broadcast football analysis: labelled football data are scarce, but pre-training already supplies robust features that generalise across the camera-angle, scale, lighting, and motion-blur variations characteristic of broadcast video.

## 2.4 Object Detection in Broadcast Football

Object detection is the task of locating and classifying objects within an image. In broadcast football the relevant objects are players, goalkeepers, referees, and the ball, and errors at this stage propagate through the rest of the pipeline: missed players degrade tracking, inaccurate boxes corrupt pitch projection, and missed ball detections



undermine possession and pass reasoning.

Formally, a detector maps an input image  $I$  to a set of predictions

$$D = \{(b_i, c_i, s_i)\}_{i=1}^N, \quad (2.2)$$

where  $b_i$  is a bounding box,  $c_i$  is the predicted class label, and  $s_i \in [0, 1]$  is a confidence score reflecting the model's certainty that an object of class  $c_i$  is present at  $b_i$ .

### 2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression

A bounding box is a rectangular region enclosing an object, parameterised either by corner coordinates or by centre, width, and height. The standard measure of localisation quality between a predicted box  $B_p$  and a ground-truth box  $B_g$  is the Intersection over Union (IoU), illustrated in Figure 2.2,

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \quad (2.3)$$

which takes values in  $[0, 1]$ , with 1 indicating perfect overlap. A prediction is counted as a true positive only if its IoU with an unmatched ground-truth box exceeds a chosen threshold and the predicted class is correct [23].

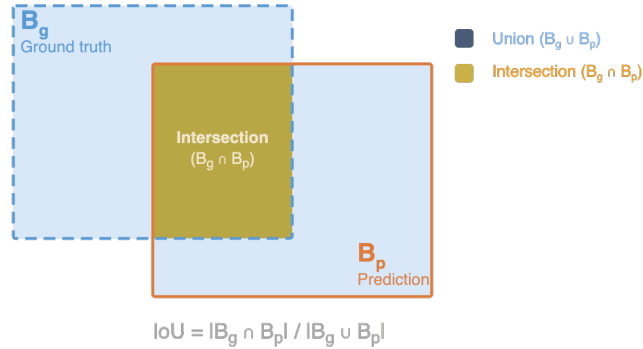


Figure 2.2: Intersection over Union (IoU) between a ground-truth bounding box  $B_g$  (blue, dashed) and a predicted bounding box  $B_p$  (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as  $\text{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$ .

Detectors typically emit several overlapping predictions for the same object. Non-Maximum Suppression (NMS) reduces these to a single detection per object via a

greedy procedure: given a candidate set  $\mathcal{B}$  of scored predictions and an overlap threshold  $\tau_{\text{NMS}}$ , at each step

$$b^* = \arg \max_{b \in \mathcal{B}} s(b),$$

the highest-scoring remaining detection  $b^*$  is retained and any  $b \in \mathcal{B}$  with  $\text{IoU}(b, b^*) \geq \tau_{\text{NMS}}$  is discarded; the procedure repeats until  $\mathcal{B}$  is empty [36]. Figure 2.3 illustrates this process. NMS matters in football footage because players cluster during tackles, set-pieces, and crowded midfield phases — without it, a single player can be counted multiple times and generate duplicate tracks downstream.

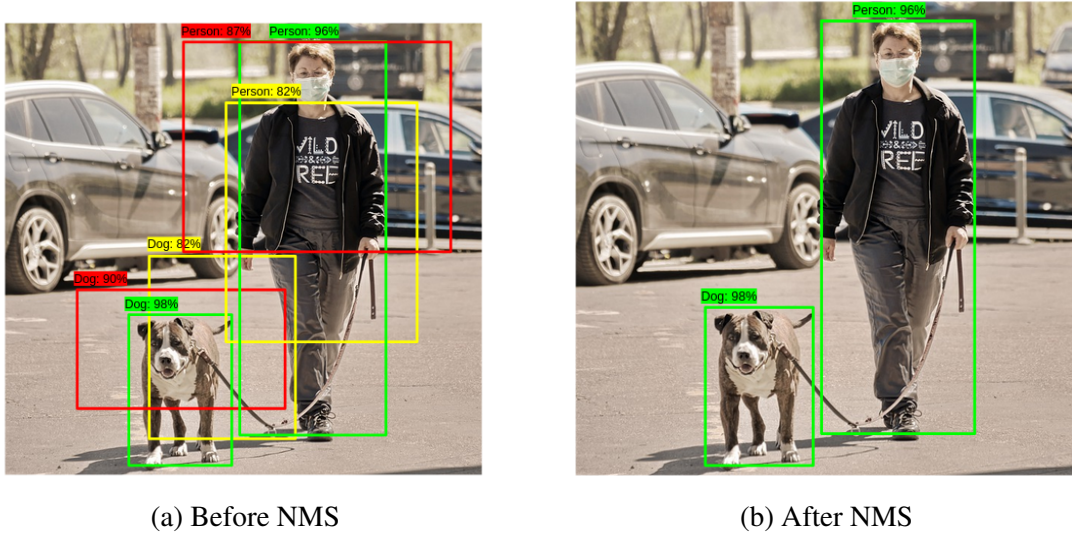


Figure 2.3: Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained.

## 2.4.2 Precision, Recall, and Mean Average Precision

Once detections have been matched to ground truth, performance is summarised by precision and recall. Precision measures how many predicted detections are correct, while recall measures how many ground-truth objects are successfully detected:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (2.4)$$

where TP, FP, and FN denote true positives, false positives, and false negatives. Varying the confidence threshold traces out a precision–recall curve: raising the threshold

improves precision at the cost of recall, and vice versa.

Average Precision (AP) summarises this curve as the area beneath it,

$$\text{AP} = \int_0^1 p(r) dr,$$

typically approximated by interpolation over a finite set of recall points. Mean Average Precision (mAP) averages AP across classes,

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c.$$

Two variants matter for this project.  $\text{mAP}_{50}$  fixes the IoU threshold at 0.50, while  $\text{mAP}_{50-95}$  averages mAP over IoU thresholds from 0.50 to 0.95 in steps of 0.05 [23] and is therefore a stricter measure of localisation. The distinction is operationally important: a loosely localised player box may suffice for visualisation but not for tracking association or pitch projection.

### 2.4.3 The YOLO Architecture

Object detectors fall broadly into two families. Two-stage detectors such as Faster R-CNN [38] first propose candidate regions and then classify and refine them, giving strong localisation at the cost of speed. Single-stage detectors such as SSD [26] and YOLO [36] predict boxes and class scores in a single forward pass, making them better suited to video-rate applications such as broadcast football analysis.

YOLO (*You Only Look Once*) treats detection as a single regression problem. The image is divided into a grid, and each cell is responsible for predicting any object whose centre falls inside it: a bounding box, an *objectness* score reflecting whether an object is present at all, and a distribution over class labels [36]. This one-shot design makes YOLO fast, but its accuracy depends critically on how the network is structured to handle objects of very different sizes — in broadcast football, a near-camera player and a far-touchline ball can differ in pixel area by two orders of magnitude.

**Architecture.** A modern YOLO-style detector consists of three stages, illustrated in Figure 2.4. A convolutional *backbone* processes the image and produces feature maps at several spatial resolutions: shallower maps preserve fine spatial detail useful for small objects, while deeper maps carry richer semantic information useful for classification. A *neck*, typically based on feature pyramid networks [22], fuses these

maps so that every prediction is informed by both fine detail and broader context. The *detection head* reads from the fused maps and outputs, at each spatial location, a bounding box, an objectness score, and class probabilities. A confidence threshold and Non-Maximum Suppression then reduce these per-location predictions to the final detection set.

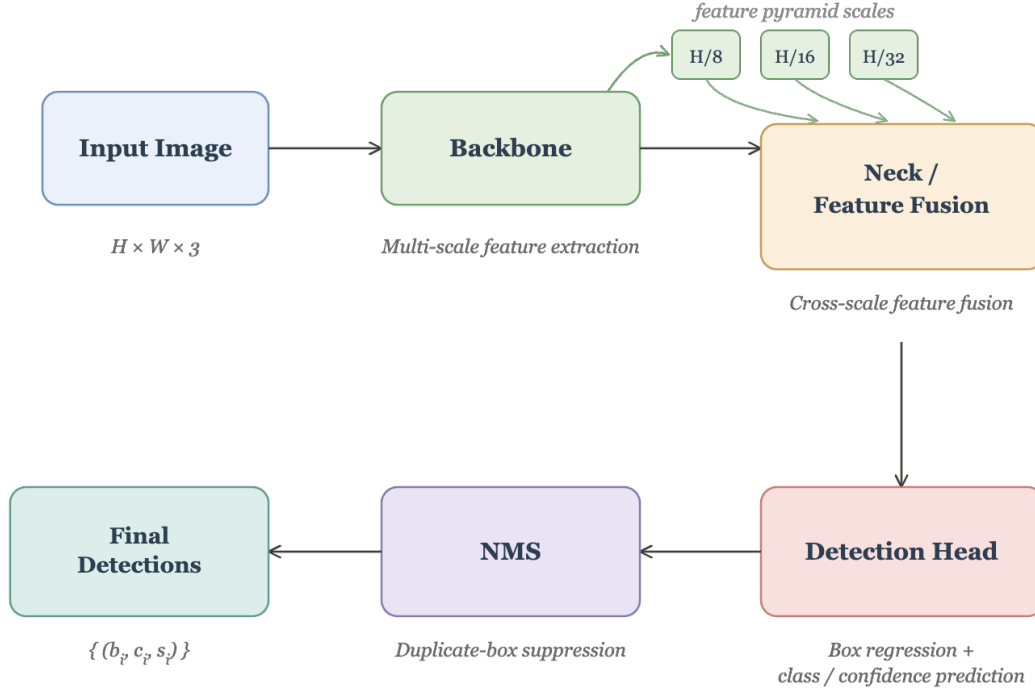


Figure 2.4: Generalised YOLO-style single-stage detection pipeline. The backbone extracts multi-scale feature maps, the neck fuses them, and the detection head predicts boxes, objectness, and class scores. NMS removes duplicate predictions to produce the final detection set  $\{(b_i, c_i, s_i)\}$ .

**How boxes are predicted.** Rather than predict absolute pixel coordinates, the detection head predicts *adjustments* relative to a known reference, which is more numerically stable to train. In the original anchor-based design, each spatial location in the feature map is associated with a set of *anchor boxes* — prior guesses of likely object shapes, with width  $p_w$  and height  $p_h$ . For an anchor at grid-cell offset  $(c_x, c_y)$ , the network predicts four numbers  $(t_x, t_y, t_w, t_h)$  that are decoded into a bounding box

$(b_x, b_y, b_w, b_h)$  by

$$\begin{aligned} b_x &= \sigma(t_x) + c_x, & b_y &= \sigma(t_y) + c_y, \\ b_w &= p_w e^{t_w}, & b_h &= p_h e^{t_h}, \end{aligned}$$

where  $\sigma$  denotes the sigmoid function [37]. The sigmoid keeps the predicted centre inside its grid cell, and the exponential keeps the predicted width and height positive while expressing them as multiplicative deviations from the anchor’s prior shape. Anchor-free variants used in YOLOv8 [16] drop the priors  $(p_w, p_h)$  altogether and instead regress the four sides of the box directly as distances from each feature-map location — a simpler design that removes a hand-tuned hyperparameter.

**Training objective.** YOLO is trained to minimise a weighted sum of three losses, one for each output of the head:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}.$$

The first term  $\mathcal{L}_{\text{box}}$  penalises localisation error, typically using an IoU-based loss such as CIOU. The second  $\mathcal{L}_{\text{obj}}$  is a binary loss over whether each predicted location actually contains an object. The third  $\mathcal{L}_{\text{cls}}$  is a classification loss over the object class. Splitting objectness from classification lets the model learn the question *is something here?* independently of *what is it?*, which helps when classes overlap visually — as players and goalkeepers do in broadcast football.

**Relevance to broadcast football.** Multi-scale feature extraction is what makes YOLO viable on broadcast footage. Player sizes span a wide range within any single frame, and the ball is often only a handful of pixels across. By combining fine- and coarse-resolution feature maps in a single network, the detector can localise objects across this entire scale range.

## 2.5 Multi-Object Tracking

Object detection produces independent bounding boxes in each frame, but does not say whether a player detected in one frame is the same player detected in the next. Multi-object tracking (MOT) addresses this by linking detections over time and assigning a persistent identity to each object [8]. Tracking is essential to this project because every downstream output — heatmaps, distance and speed estimates, team assignment,

possession reasoning, and offside analysis — requires following the same player across frames.

Formally, let

$$D_t = \{d_t^1, \dots, d_t^{m_t}\} \quad (2.5)$$

denote the detections at frame  $t$ , each carrying a bounding box, class, and confidence, and let

$$\mathcal{T}_{t-1} = \{\tau_{t-1}^1, \dots, \tau_{t-1}^{m_{t-1}}\} \quad (2.6)$$

denote the active tracks from the previous frame. Tracking must assign each  $d \in D_t$  to a track in  $\mathcal{T}_{t-1}$ , create new tracks for unmatched detections, and gracefully handle tracks for which no matching detection arrives.

### 2.5.1 Tracking-by-Detection

Most modern MOT systems follow the *tracking-by-detection* paradigm [4]: an object detector is applied independently to each frame, and a separate tracker associates the resulting detections over time. This decouples visual recognition from temporal association — the detector answers *what is in this frame?* while the tracker answers *which detections belong to the same object?* The framework is modular (detector improvements can be dropped in without redesigning the tracker) and online (associations are produced frame-by-frame, without waiting for the full sequence). Its principal weakness is detector dependence: missed detections fragment tracks, false positives spawn spurious tracks, and inaccurate boxes produce incorrect associations — failures that cluster in crowded scenes such as set-pieces and midfield contests.

### 2.5.2 Motion Prediction and Data Association

A standard motion model is the Kalman filter, which recursively estimates the state of a linear-Gaussian system from noisy observations [17]. For a tracked bounding box, a common state under a constant-velocity assumption is

$$\mathbf{x}_t = [x_t, y_t, w_t, h_t, \dot{x}_t, \dot{y}_t, \dot{w}_t, \dot{h}_t]^\top,$$

combining the box centre, its width and height, and their rates of change. The filter alternates between two steps. The *predict* step propagates the previous state ( $\mathbf{x}_{t-1}$ ) and

its covariance ( $P_{t-1}$ ) forward,

$$\hat{\mathbf{x}}_t^- = F\mathbf{x}_{t-1}, \quad P_t^- = FP_{t-1}F^\top + Q, \quad (2.7)$$

where  $F$  is the state-transition matrix encoding the motion model and  $Q$  is the process-noise covariance. The predicted state  $\hat{\mathbf{x}}_t^-$  gives the expected location of the track before seeing the detector output for frame  $t$ . The *update* step then corrects this prediction using a matched detection  $\mathbf{z}_t$ :

$$K_t = P_t^- H^\top (HP_t^- H^\top + R)^{-1}, \quad (2.8)$$

$$\mathbf{x}_t = \hat{\mathbf{x}}_t^- + K_t(\mathbf{z}_t - H\hat{\mathbf{x}}_t^-), \quad P_t = (I - K_t H)P_t^-, \quad (2.9)$$

where  $H$  is the observation matrix,  $R$  is the measurement-noise covariance, and the Kalman gain  $K_t$  weighs the new evidence against the prior prediction, trusting the detection more when measurement noise is small relative to predicted state uncertainty. The term  $\mathbf{z}_t - H\hat{\mathbf{x}}_t^-$  is the difference between the observed detection and the predicted measurement.

Given predicted track locations, association reduces to a bipartite matching problem in which each track receives at most one detection and vice versa. We construct a cost matrix  $C \in \mathbb{R}^{m \times n}$  with

$$C_{ij} = 1 - \text{IoU}(\hat{B}_i, B_j), \quad (2.10)$$

where  $\hat{B}_i$  is the predicted box for track  $i$  and  $B_j$  is the box of detection  $j$ ; lower cost means better match. The Hungarian algorithm solves this assignment problem to optimality in  $O(n^3)$  time [20]. Unmatched detections seed new tracks; unmatched tracks are buffered and eventually deleted if not recovered. This Kalman–Hungarian skeleton underlies SORT [4], ByteTrack [46], and BoT-SORT [2].

### 2.5.3 ByteTrack

ByteTrack [46] observes that low-confidence detections are not all false: confidence frequently drops during occlusion, motion blur, or truncation while the object remains present. Discarding them therefore loses useful evidence. ByteTrack instead partitions detections by confidence,

$$D_t^{\text{high}} = \{d \in D_t : s_d \geq \tau_{\text{high}}\}, \quad D_t^{\text{low}} = \{d \in D_t : s_d < \tau_{\text{high}}\}, \quad (2.11)$$

where  $s_d$  is the confidence score of detection  $d$ ,  $\tau_{\text{high}}$  is the high-confidence threshold, and runs association in two passes (Figure 2.5). Tracks are first matched against  $D_t^{\text{high}}$  using the Hungarian algorithm with IoU cost; any track that remains unmatched is then given a second chance against  $D_t^{\text{low}}$ . High-confidence detections that fail to match seed new tracks; tracks unmatched after both passes are held in a buffer for a short period before deletion. This design preserves track continuity through short occlusions and detector dropouts while still anchoring associations to high-confidence evidence.

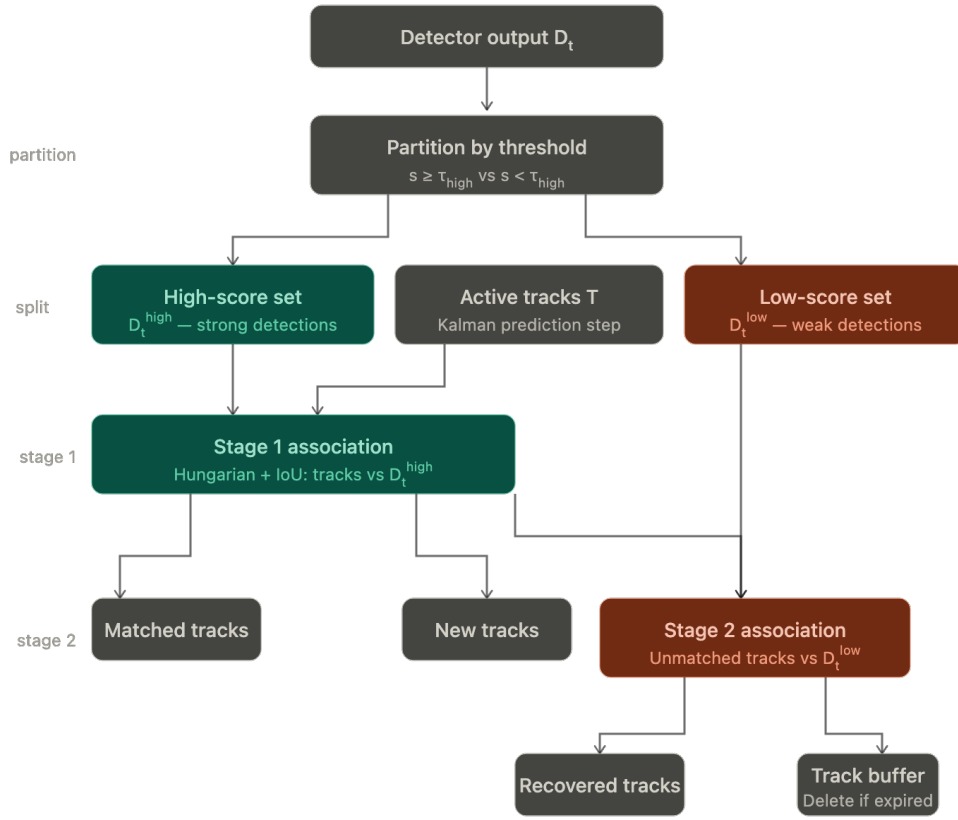


Figure 2.5: ByteTrack two-stage association process. Detector output is partitioned by confidence threshold  $\tau_{\text{high}}$  into a high-score set  $D_t^{\text{high}}$  and a low-score set  $D_t^{\text{low}}$ . Stage 1 matches active tracks against  $D_t^{\text{high}}$  using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against  $D_t^{\text{low}}$ . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames.



### 2.5.4 BoT-SORT

BoT-SORT [2] extends the tracking-by-detection skeleton with two refinements aimed at challenging video. First, *camera-motion compensation*: when the camera pans, tilts, or zooms, all bounding-box positions shift together, and naïve IoU between predicted and observed boxes becomes unreliable. BoT-SORT estimates a global affine transform between consecutive frames — typically by matching sparse keypoints in the static background — and pre-warps each predicted track location into the current frame’s coordinates before association. Second, *appearance-based re-identification*: visual features extracted from detection crops disambiguate spatially similar candidates, particularly during occlusions. Both refinements add per-frame computational cost relative to ByteTrack.

### 2.5.5 Tracking Evaluation: HOTA

Tracking quality must capture two properties simultaneously: how accurately objects are localised, and how consistently their identities persist over time. Multi-Object Tracking Accuracy (MOTA) and Identification F1 (IDF1) are established baselines [8], but both conflate detection and association in ways that obscure the source of errors.

Higher Order Tracking Accuracy (HOTA) addresses this by decomposing performance into separate detection and association scores [28]:

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha}, \quad (2.12)$$

where  $\text{DetA}_\alpha$  measures box overlap with ground truth at localisation threshold  $\alpha$ , and  $\text{AssA}_\alpha$  measures identity consistency — the average proportion of ground-truth and predicted trajectories that share an identity over their joint lifetime. The geometric mean ensures that neither component can dominate: a tracker that localises well but fragments identities, or associates well but misses many objects, will be penalised in both factors. The final HOTA score averages Equation 2.12 over  $\alpha \in [0.05, 0.95]$ , giving a single threshold-independent summary.

## 2.6 Team Identification from Visual Features

Object detection and tracking determine where players are and how they move, but neither directly recovers which team each player belongs to. Team identity is not

produced by a generic detector, and manually labelling every player per match would defeat the purpose of an automated pipeline. Because players on the same team share a visually similar kit, the task is naturally cast as an unsupervised grouping problem on player appearance [15].

### 2.6.1 Team Identification as Clustering

Each detected player crop  $I_i$  is mapped to a feature vector

$$\mathbf{z}_i = f(I_i), \quad \mathbf{z}_i \in \mathbb{R}^d, \quad (2.13)$$

by a feature extractor  $f$  chosen so that crops of players in the same kit produce similar vectors and crops of opposing players produce dissimilar ones. The team-identification problem is then to partition  $\{\mathbf{z}_i\}_{i=1}^N$  into groups corresponding to the teams on the pitch — a standard unsupervised clustering problem [29]. For the two outfield teams the natural choice is two clusters, since their kits are required by competition rules to be visually distinguishable.

### 2.6.2 Visual Representations

The classical approach extracts colour histograms in spaces such as RGB, HSV, or Lab and clusters players by histogram similarity [15]. Such features are fast, interpretable, and align with the cue humans use, but they are sensitive to lighting changes, shadows, and background pixels included in the crop, and they discard spatial structure entirely.

A more robust alternative is to use deep visual embeddings. Modern pretrained encoders such as CLIP, SigLIP, and DINOv2 learn transferable visual representations from internet-scale pre-training [35, 45, 33] and produce embeddings that capture colour, texture, shape, and pattern jointly in a single vector. Because their training data covers extensive visual variation, the resulting features generalise to new domains — including broadcast football crops — without any domain-specific fine-tuning, which makes them well-suited to a setting in which match-specific labels are unavailable.

For clustering, embeddings  $(\mathbf{z}_i)$  are typically L2-normalised so that

$$\tilde{\mathbf{z}}_i = \mathbf{z}_i / \|\mathbf{z}_i\|_2,$$

and similarity is measured by cosine similarity,

$$\text{sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\|_2 \|\mathbf{z}_j\|_2}.$$

On the unit sphere, cosine similarity and squared Euclidean distance differ only by a constant —  $\|\tilde{\mathbf{z}}_i - \tilde{\mathbf{z}}_j\|_2^2 = 2 - 2\tilde{\mathbf{z}}_i^\top \tilde{\mathbf{z}}_j$  — so Euclidean-distance clustering on normalised embeddings is equivalent to clustering by cosine similarity. This makes pretrained embeddings directly usable with the distance-based clustering algorithms below.

### 2.6.3 Dimensionality Reduction

Deep embeddings are typically high-dimensional ( $d \in [512, 1024]$  for the encoders cited above), and distance-based clustering becomes less stable in such spaces because pairwise distances concentrate — many points appear nearly equidistant, a phenomenon known as the curse of dimensionality [1]. Dimensionality reduction maps embeddings into a lower-dimensional space  $\mathbb{R}^{d'}$  with  $d' \ll d$  before clustering:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, \quad \mathbf{y}_i = \phi(\mathbf{z}_i). \quad (2.14)$$

Principal Component Analysis (PCA) is the classical linear choice. Given centred embeddings stacked as  $Z \in \mathbb{R}^{N \times d}$ , PCA computes the eigendecomposition of the sample covariance matrix  $\Sigma = \frac{1}{N} Z^\top Z$  and projects onto the top- $d'$  eigenvectors of  $\Sigma$ , yielding the linear subspace that maximises preserved variance. PCA is fast and deterministic but its linearity means it cannot unfold curved manifold structure that embedding models often produce.

UMAP (Uniform Manifold Approximation and Projection) is a non-linear alternative that constructs a weighted neighbourhood graph in the high-dimensional space and optimises a low-dimensional layout to preserve its local topology [32]. The objective minimises the cross-entropy between the high- and low-dimensional neighbour distributions, which preserves the local similarity structure that matters for team identification: players in the same kit should remain close, even when their global arrangement on a manifold is non-linear.

### 2.6.4 $k$ -Means Clustering

Given reduced features  $\{\mathbf{y}_i\}_{i=1}^N$ ,  $k$ -means partitions them into  $k$  clusters by minimising the within-cluster sum of squared distances [29]:

$$\min_{\{\mu_j\}_{j=1}^k} \sum_{i=1}^N \min_{1 \leq j \leq k} \|\mathbf{y}_i - \mu_j\|_2^2, \quad (2.15)$$

where  $\mu_j$  is the centroid of cluster  $j$ . This objective is non-convex, so direct minimisation is intractable. In practice it is solved by Lloyd’s algorithm, which alternates two steps until convergence. Given current centroids  $\{\mu_j\}$ , each point is assigned to its nearest centroid,

$$a_i = \arg \min_{1 \leq j \leq k} \|\mathbf{y}_i - \mu_j\|_2^2,$$

and each centroid is then updated as the mean of the points assigned to it,

$$\mu_j = \frac{1}{|\{i : a_i = j\}|} \sum_{i: a_i = j} \mathbf{y}_i.$$

Each step strictly decreases or preserves the objective, guaranteeing convergence to a local minimum. The local optimum reached depends on initialisation;  $k$ -means++ provides a probabilistic seeding scheme that selects initial centroids far apart in feature space, giving provably better expected performance than uniform random initialisation. For team identification on outfield players  $k = 2$ , and the algorithm recovers the two groups up to an arbitrary label permutation — separating *which player is on which team*, but not *which cluster corresponds to which named team*.

## 2.7 Pitch Registration and Planar Homography

Detections and tracks are produced in image coordinates, but tactical analysis — bird’s-eye visualisation, heatmaps, distance and speed estimation, possession analysis, offside reasoning — requires positions in a common pitch coordinate system. Pitch registration estimates a geometric mapping between the broadcast image and a canonical top-down pitch model, allowing image-space observations to be projected into pitch space [42, 41].

### 2.7.1 Planar Homography

The football pitch is approximately planar, and projective geometry tells us that any two views of a plane are related by a  $3 \times 3$  projective transformation called a *planar homography* [14]. Working in homogeneous coordinates, an image point  $\tilde{\mathbf{x}} = (u, v, 1)^\top$  and its corresponding pitch-space point  $\tilde{\mathbf{X}} = (x, y, 1)^\top$  are related by

$$\tilde{\mathbf{X}} \sim H\tilde{\mathbf{x}}, \quad (2.16)$$

where  $\sim$  denotes equality up to a non-zero scale factor. Because  $H$  is defined only up to scale, it has eight degrees of freedom rather than nine. Writing

$$H = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix},$$

the explicit pitch coordinates of an image point are

$$x = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}, \quad y = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}, \quad (2.17)$$

which makes the projective nature explicit: the denominator is what produces perspective foreshortening, and reduces to an affine transformation only when  $h_{31} = h_{32} = 0$ .

The mapping is valid only for points lying on (or very near) the pitch plane. Player feet are therefore the correct reference point for projection, and the bottom centre of each player's bounding box serves as a practical approximation to the foot location.

### 2.7.2 Estimating $H$ from Point Correspondences

Estimating  $H$  requires correspondences  $\{(\mathbf{x}_i, \mathbf{X}_i)\}_{i=1}^N$  between image and pitch points. Each correspondence yields *two* independent constraints on  $H$ : from Equation 2.16, the cross product  $\tilde{\mathbf{X}}_i \times H\tilde{\mathbf{x}}_i = \mathbf{0}$  gives three scalar equations, of which only two are linearly independent. Since  $H$  has eight degrees of freedom, at least four non-collinear correspondences are required to determine it; with more than four, the system becomes overdetermined.

The standard solution is the Direct Linear Transform (DLT) algorithm [14]. Stacking the two constraints per correspondence produces a homogeneous linear system

$$A\mathbf{h} = \mathbf{0}, \quad A \in \mathbb{R}^{2N \times 9}, \quad (2.18)$$

where  $\mathbf{h} \in \mathbb{R}^9$  is the vectorised form of  $H$ . The least-squares solution under the constraint  $\|\mathbf{h}\|_2 = 1$  is the right singular vector of  $A$  corresponding to its smallest singular value, obtained from the singular value decomposition of  $A$ . In practice, image and pitch coordinates are first normalised — typically translated to have zero mean and scaled to have unit average distance from the origin — to improve numerical conditioning [14] before solving.

Real correspondences contain both noise and outliers — incorrectly localised keypoints whose error is large enough to corrupt the least-squares estimate. Robust estimation is therefore performed using RANSAC, which repeatedly samples minimal sets of four correspondences, fits a candidate homography from each, and counts how many of the remaining correspondences are consistent with it (within a chosen reprojection threshold). The homography with the largest consensus set is retained, and the final estimate is refined by re-running DLT on its inliers only. RANSAC tolerates outlier rates well above 50%, which makes it the standard choice when correspondences come from an automatic detector rather than from manual annotation.

### 2.7.3 Pitch Keypoint Detection

The correspondences required by DLT are obtained by detecting semantic landmarks on the playing surface — line intersections, penalty-area corners, centre-circle tangencies, and similar distinctive points formed by pitch markings — whose pitch-space coordinates are fixed by the laws of the game and therefore known a priori. Detecting these landmarks in the broadcast image yields exactly the  $(\mathbf{x}_i, \mathbf{X}_i)$  pairs that Equation 2.18 requires.

This is structurally a keypoint detection problem and is solved by the same family of CNN architectures used for human pose estimation, with the target points replaced by fixed pitch landmarks. A convolutional backbone extracts multi-scale feature maps and a prediction head outputs per-landmark confidences and locations [16]. Keypoint quantity, accuracy, and spatial spread all matter: more correspondences overdetermine the system and average out localisation noise; correspondences distributed across the visible pitch constrain the transformation more tightly than correspondences clustered in one region.

### 2.7.4 Reprojection Error

Registration quality is measured by reprojection error. For correspondence  $i$ ,

$$e_i = \|\mathbf{X}_i - \pi(H\tilde{\mathbf{x}}_i)\|_2, \quad (2.19)$$

where  $\pi(\cdot)$  converts homogeneous coordinates back to Cartesian by dividing by the third component. The mean reprojection error over  $N$  correspondences,

$$\bar{e} = \frac{1}{N} \sum_{i=1}^N e_i, \quad (2.20)$$

summarises how closely the estimated  $H$  maps the detected landmarks back to their known pitch locations.

Low reprojection error is necessary but not sufficient for accurate downstream projection. The metric is computed only on the keypoints used to fit  $H$ , so a homography that fits its inputs perfectly can still extrapolate poorly to regions of the pitch where no landmarks were detected — particularly when keypoints cluster in one half of the image, leaving the other half geometrically underconstrained.

## 2.8 Spatial and Tactical Analysis in Pitch Space

Once detections and tracks have been mapped to a canonical pitch model, analysis can proceed in a coordinate system that is independent of the broadcast camera. This matters because pixel distances do not correspond uniformly to real-world distances under projective viewing: equal displacements in image space can encode very different real displacements depending on where they occur in the frame. Pitch-space coordinates remove this distortion, making it possible to compare positions, compute movement statistics, and reason about tactical events in a common geometric frame [42, 11].

### 2.8.1 Pitch-Space Trajectories

Each tracked player must be reduced to a single image point before the homography is applied. Any such point must lie on the ground plane, since the homography is valid only there; an image point above the plane projects incorrectly because its line of sight does not intersect the pitch where the player stands.

Let  $\mathbf{q}_t^{(i)} \in \mathbb{R}^2$  be the chosen ground-plane image point for player  $i$  at frame  $t$ . Given

an image-to-pitch homography  $H$ , the projected pitch-space position is

$$\mathbf{p}_t^{(i)} = \pi\left(H\tilde{\mathbf{q}}_t^{(i)}\right), \quad (2.21)$$

where  $\tilde{\mathbf{q}}_t^{(i)}$  is the homogeneous form of the image point and  $\pi(\cdot)$  converts back to Cartesian coordinates. The sequence  $\{\mathbf{p}_t^{(i)}\}_{t=1}^T$  is the pitch-space trajectory of player  $i$ , and is the geometric primitive on which all subsequent spatial analysis is built.

## 2.8.2 Heatmap Construction

A heatmap summarises the spatial distribution of a player's or team's positions over time. The pitch is divided into a regular grid, and each projected position contributes to the cell containing it:

$$H_i(u, v) = \sum_{t=1}^T \mathbf{1}\left[(u, v) = \text{bin}\left(\mathbf{p}_t^{(i)}\right)\right], \quad (2.22)$$

where  $\mathbf{1}[\cdot]$  is the indicator function and  $\text{bin}(\cdot)$  maps a continuous pitch coordinate to a discrete grid cell. Equation 2.22 is a two-dimensional histogram of the trajectory, and a team-level heatmap is obtained by accumulating positions from all players assigned to that team.

Histogram estimates are noisy, particularly when sample counts per cell are small. Smoothing with an isotropic Gaussian kernel,

$$\tilde{H}_i = H_i * G_\sigma, \quad (2.23)$$

yields a kernel density estimate of the underlying spatial distribution. The bandwidth  $\sigma$  controls the bias–variance trade-off: a larger  $\sigma$  suppresses noise but blurs spatial detail; a smaller  $\sigma$  preserves detail at the cost of higher variance per cell.

## 2.8.3 Speed and Distance Estimation

Movement statistics derive from temporal differences of the trajectory. With a video frame rate  $f_s$  and inter-frame interval  $\Delta t = 1/f_s$ , for player  $i$ , a finite-difference velocity estimate at frame  $t$  is

$$\mathbf{v}_t^{(i)} = \frac{\mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)}}{\Delta t}, \quad (2.24)$$



where  $\mathbf{p}_t^{(i)}$  and  $\mathbf{p}_{t-1}^{(i)}$  are consecutive pitch-space positions. The corresponding instantaneous speed is

$$s_t^{(i)} = \|\mathbf{v}_t^{(i)}\|_2. \quad (2.25)$$

Total distance covered can then be approximated by summing the Euclidean displacement between consecutive positions:

$$d^{(i)} = \sum_{t=2}^T \left\| \mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)} \right\|_2. \quad (2.26)$$

Finite differencing is intrinsically noise-amplifying. If each pitch-space position carries independent Gaussian localisation noise with variance  $\sigma_p^2$ , the variance of Equation 2.24 is  $2\sigma_p^2/\Delta t^2$ , so a small per-frame localisation error becomes a large per-frame velocity error. This is a property of the differencing operator itself rather than of any particular pipeline.

## 2.8.4 Possession via Nearest-Player Assignment

Possession and passes are not detected directly from a frame. They are inferred from the spatial relationship between the ball, the players, and their team identities over time. The simplest model assigns the ball to the spatially closest eligible player. Given a ball position  $\mathbf{b}_t \in \mathbb{R}^2$  and a candidate set  $\mathcal{A}_t$  of players' pitch positions  $\mathbf{p}_t^{(i)}$ , the assigned holder is

$$h_t = \arg \min_{i \in \mathcal{A}_t} \|\mathbf{b}_t - \mathbf{p}_t^{(i)}\|_2. \quad (2.27)$$

This is exactly the assignment of  $\mathbf{b}_t$  to a cell of the Voronoi diagram of  $\{\mathbf{p}_t^{(i)}\}$ , in which each player owns the region of pitch closer to them than to any other player. A pass corresponds to a transition of  $h_t$  between distinct players, normally on the same team.

A more refined formulation is *pitch control*, in which each player exerts a continuous influence over the pitch as a function of position, velocity, and orientation, and the ball is assigned probabilistically to the player with the greatest influence at its current location [10]. Each player  $i$  is associated with an influence function  $I^{(i)}(\mathbf{r}, t)$  — typically a bivariate Gaussian whose mean is offset forward of the player along their velocity vector and whose covariance grows with speed — and the probability that player  $i$  controls location  $\mathbf{r}$  at time  $t$  is

$$P^{(i)}(\mathbf{r}, t) = \frac{I^{(i)}(\mathbf{r}, t)}{\sum_j I^{(j)}(\mathbf{r}, t)}.$$

Pitch control captures dynamic context that nearest-player assignment ignores — momentum, reach, time-to-arrive — but requires reliable velocities and orientations, and therefore higher-quality input than nearest-player assignment.

### 2.8.5 Offside Rule Formalisation

Offside is fundamentally a spatial rule and therefore admits a pitch-space formalisation. Under Law 11 of the Laws of the Game, a player is in an offside position if, at the moment the ball is played by a team-mate, they are in the opponents’ half and nearer to the opponents’ goal line than both the ball and the second-last opponent [40].

Choose a coordinate system in which the attacking direction of the team in possession is the positive  $x$ -axis. Let  $x_t^{(a)}$ ,  $x_t^{(b)}$ , and  $x_t^{(o_2)}$  be the longitudinal coordinates at the moment of the pass of an attacking player, the ball, and the second-last opponent respectively. The positional offside test is then

$$x_t^{(a)} > \max\left(x_t^{(b)}, x_t^{(o_2)}\right), \quad (2.28)$$

combined with the requirement that the attacker is in the opponents’ half. The maximum expresses the rule’s “beyond both” condition: the second-last opponent serves as the defensive line, but the ball overrides this when it is ahead of that line.

Equation 2.28 captures positional offside only. Law 11 additionally requires the player to become actively involved in play before being penalised, which is a behavioural rather than purely spatial condition [40]; this falls outside what pitch-space geometry alone can decide. The formalisation nonetheless illustrates how a tactical rule reduces to a geometric inequality once players, ball, and pitch share a coordinate system.

## 2.9 Chapter Summary

This chapter has developed the theoretical foundation for the broadcast football analysis pipeline. The unifying view is that the system is a composition of stages — detection, tracking, team identification, pitch registration, and pitch-space tactical reasoning — each formalised mathematically and each consuming the outputs of those before it. This structure makes the pipeline modular and individually testable, but also tightly coupled: errors at any stage propagate downstream, so upstream quality bounds what later stages can achieve. The methods, metrics, and notation introduced here are

applied directly in the implementation described in Chapter 3 and in the evaluation presented in Chapter 5.

# Chapter 3

## Methodology and Implementation

### 3.1 Chapter Overview

Chapter 2 surveyed the methods relevant to each stage of a broadcast football analysis pipeline—object detection, tracking, team identification, pitch registration, and the derivation of tactical outputs—without committing to a specific realisation. This chapter moves from that theoretical grounding to the concrete system developed for this project, describing the architecture and implementation of each stage and the reasoning behind the choices made at each one.

The pipeline takes a single broadcast video stream as input and processes it through a sequence of stages: player, goalkeeper, referee and ball detection with dedicated YOLO models; multi-object tracking by association; team identification via SigLIP embeddings of torso crops followed by dimensionality reduction and clustering; pitch registration from keypoint correspondences resolved into a planar homography; and projection of tracked players into a pitch-coordinate frame from which heatmaps, speed and distance, possession, and offside geometry are derived. Each stage consumes the output of those preceding it, and the chapter follows this same ordering so that each design decision can be read in the context of the inputs it must handle and the outputs it must produce.

For each stage, the chapter justifies the design choice against the alternatives surveyed in Chapter 2, specifies parameter values and algorithmic details in sufficient detail for a peer to reproduce the work, and presents illustrative outputs from the implemented system. The datasets used for training the bespoke detection and keypoint models are described, together with the training procedure and the resulting model artefacts. The chapter also defines the evaluation methodology that will be applied in

Chapter 5, setting out the metrics, test data, and protocols by which each pipeline stage and the system as a whole will be assessed.

Chapter 4 presents the results obtained by running the pipeline on evaluation data, and Chapter 5 discusses those results and their implications; the present chapter is concerned with how the system was built and why, not with its measured performance.

## 3.2 Requirements and Design Objectives

Before the system architecture is presented, this section sets out the requirements the pipeline is designed to meet. Functional requirements define what the system must do; non-functional requirements define the qualities it should exhibit; and success criteria state the measurable thresholds against which each component will be judged in Chapter 5. The requirements derive from the project aims set out in Chapter 1 and are framed here in terms suited to system design, so that the architectural and implementation choices described in the remainder of this chapter can be read against them.

### 3.2.1 Functional Requirements

The functional requirements describe the end-to-end behaviour of the pipeline. The system takes as input a single broadcast football video and produces, for each frame, a set of per-object detections, persistent track identities, team labels, pitch-coordinate positions, and a collection of derived tactical outputs. The requirements are decomposed by pipeline stage so that each subsequent section of this chapter can be mapped directly to the requirement it satisfies.

**FR1: Object detection.** The system shall detect the four classes relevant to broadcast football analysis—players, goalkeepers, referees, and the ball—in each frame of the input video, returning axis-aligned bounding boxes and class labels. Given the extreme scale disparity between outfield players and the ball, detection of the ball shall be handled by a dedicated model operating at the same input resolution but with its own training regime and confidence regime, rather than relying on the generic detector alone.

**FR2: Multi-object tracking.** The system shall maintain consistent identities for detected players, goalkeepers, and referees across frames, associating detections over

time into persistent tracks. Tracker choice shall be configurable so that alternative association strategies can be compared. Ball identity is not required to persist, since the ball is a single object and its per-frame localisation is sufficient for downstream use.

**FR3: Team identification.** The system shall assign each tracked player to one of two teams without manual labels, using visual appearance alone. The assignment shall be stable over time: transient misclassification on a single frame shall not be allowed to propagate as the team label returned for a track. Goalkeepers, whose kits typically differ from their outfield teammates, shall be resolved to a team by a separate mechanism rather than being forced into the outfield clustering.

**FR4: Pitch registration.** The system shall recover the planar homography between the broadcast image and a canonical pitch model, permitting image-plane coordinates to be mapped to pitch-plane coordinates. Registration shall tolerate the fact that the full pitch is rarely visible in a broadcast frame, and shall degrade gracefully when insufficient correspondences are available in a given frame by reusing the most recent valid transform until a new valid estimate is obtained.

**FR5: Tactical output generation.** From the projected per-track trajectories the system shall derive a set of tactical outputs: per-player heatmaps of pitch occupation, speed and distance statistics, team possession, pass events, and offside candidate frames. Each output shall be produced in pitch coordinates, so that the reported quantities are physically meaningful rather than tied to the broadcast view.

**FR6: Visualisation and persistence.** The system shall render the principal outputs for inspection—annotated broadcast video with detection, track, and team overlays; a bird’s-eye projection of tracked objects onto the pitch model; and rendered heatmaps—and shall persist these together with any per-frame numerical outputs so that results can be reviewed offline and used as input to the evaluation procedure.

### 3.2.2 Non-functional Requirements

The non-functional requirements describe qualities the system should exhibit irrespective of the specific outputs it produces.

**NFR1: Reproducibility.** The full configuration required to reproduce a given run—model weights, parameter values, and preset selections—shall be recoverable from a single set of launch arguments, and the training of any bespoke models shall be recorded with sufficient detail (dataset, hyperparameters, training curves) to be repeated by a peer.

**NFR2: Modularity.** Each pipeline stage shall be implemented as a self-contained component with a clearly defined input and output contract, so that an individual stage can be replaced, reconfigured, or evaluated in isolation without requiring changes to the surrounding system.

**NFR3: Configurability.** Parameter values that govern behaviour at runtime—detection thresholds, tracker selection, inference strides, warmup sizes, smoothing windows, displacement gates—shall be exposed through the command-line interface rather than embedded in source code, and reasonable defaults shall be provided in a single location so that the system can be run without a long argument list.

**NFR4: Robustness to broadcast artefacts.** The pipeline operates on broadcast video rather than instrumented capture, and must therefore tolerate partial occlusion, motion blur, pan-tilt-zoom camera motion, and the appearance of on-screen graphics without catastrophic failure. Robustness is treated as a design objective at each stage rather than as a single global property, with appropriate mechanisms (temporal smoothing, displacement gates, majority-vote filtering) introduced where individual stages are known to be sensitive.

**NFR5: Graceful degradation.** When an individual stage cannot produce a usable output for a given frame, the pipeline shall continue to operate on subsequent frames rather than terminate. For pitch registration in particular, the system shall retain the most recent valid homography and reuse it until a new valid estimate is produced. The extent to which other stages meet this objective is discussed alongside the relevant component.

**NFR6: Inspectability.** Intermediate outputs of the pipeline—raw detections, tracked identities, projected positions, registration overlays, per-track statistics—shall be explicable for inspection, so that failure modes can be localised to specific stages during

development and evaluation, and so that the evaluation procedure can assess each stage independently.

**NFR7: Runtime.** The pipeline is intended for offline post-match analysis rather than real-time or live operation. Runtime is therefore not a primary design objective: the system shall complete processing of a broadcast clip on a single consumer GPU within a reasonable working session, but no throughput target is set, and design decisions throughout the pipeline prioritise per-stage accuracy and inspectability over end-to-end speed. The measured processing cost and the stages that dominate it are discussed alongside the results in Chapter 5.

### 3.2.3 Success Criteria

The success criteria restate, in measurable form, the conditions under which each functional requirement is considered satisfactorily met. Specific numerical thresholds are stated where they derive from the project aims set in Chapter 1; the full evaluation protocols by which these are assessed are given in Section 3.10 and applied in Chapter 5.

**SC1 (FR1, Detection).** The generic object detector shall achieve a mean average precision of at least on the held-out test split, with per-class performance reported for player, goalkeeper, and referee. The dedicated ball detector shall achieve a per-frame recall of at least under the confidence regime used at inference, at a false-positive rate that the temporal-filtering stage (Section ??) can reasonably suppress.

[target  
mAP@0.5  
from  
Ch1]

**SC2 (FR2, Tracking).** Tracking performance shall be reported using the HOTA family of metrics [28], with a target HOTA of at least on the evaluation clips. Identity switches shall be reported alongside and kept below per minute of broadcast time.

[target]

[target]

[target]

**SC3 (FR3, Team identification).** The per-frame team assignment shall agree with manually annotated ground truth on at least of sampled player crops, and the per-track label returned after temporal smoothing shall agree on at least of tracks. Goalkeeper assignment shall be correct on at least of goalkeeper tracks.

[target]

[target]

[target]

**SC4 (FR4, Registration).** Pitch registration shall achieve a mean reprojection error of no more than on evaluation frames where ground-truth correspondences are available, and shall produce a valid homography on at least of frames in a continuous

[target,  
in  
pitch  
me-  
tres]

[target]



broadcast clip.

**SC5 (FR5, Tactical outputs).** Tactical outputs shall be plausible when inspected against the broadcast source: heatmaps shall reflect known player roles, speed and distance values shall fall within physically reasonable ranges reported in the sports-science literature, possession totals shall be consistent with a human annotator’s reading of the clip, detected pass events shall match annotated passes with a precision and recall of at least , and offside candidate frames shall agree with a human referee’s reading.

[target]

**SC6 (End-to-end usefulness).** When run end-to-end on unseen broadcast footage, the system shall produce the full set of visualisations and statistics described under FR6 without manual intervention, and the resulting outputs shall be informative enough that a non-specialist viewer can interpret the match in tactical terms from the produced artefacts alone.

## 3.3 System Architecture

This section describes the architecture of the pipeline at a level that permits the work to be reproduced by a peer. The pipeline is presented first as a structural overview, showing how the stages are composed and where models, rule-based logic, and caching enter the design (Section 3.3.1). The data objects passed between stages are then specified with their types, shapes, and provenance (Section 3.3.2), and the user-facing outputs of the system are catalogued (Section 3.3.3). The algorithmic content of each stage is deferred to Sections 3.6 through ??, which describe how the stages are realised and why.

### 3.3.1 Pipeline Overview

Figure 3.1 shows the end-to-end architecture. A single broadcast video frame is processed through two parallel branches that converge at a pitch-space projection stage, after which all higher-level analytics operate in a shared pitch coordinate frame rather than in the image plane.

The left branch handles *what* is in the frame: objects are detected, outfield actors are tracked over time, and each tracked player is assigned to a team. The right branch

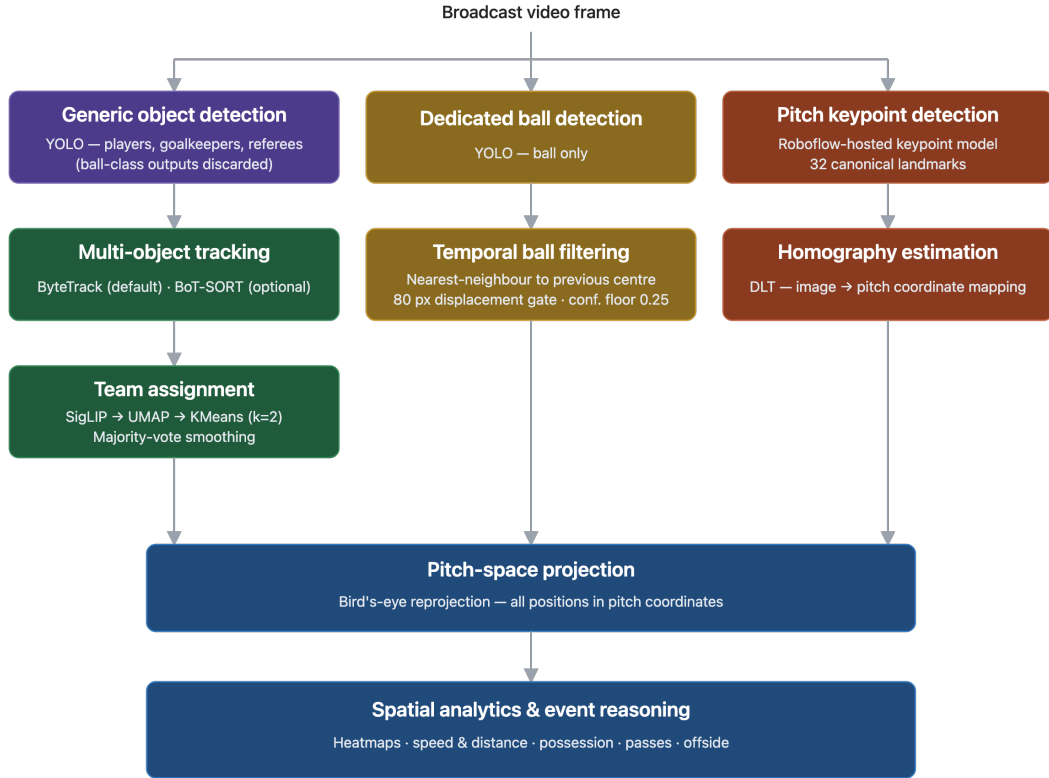


Figure 3.1: End-to-end pipeline architecture. The broadcast frame enters at the top and is processed through two parallel branches—appearance-based detection and tracking on the left, pitch-geometry recovery on the right—which converge at the pitch-space projection stage. All higher-level tactical analytics operate in pitch coordinates. Section numbers on the left indicate where each stage is described in detail.

handles *where* the frame sits in pitch-space: canonical landmarks on the pitch are detected and resolved into a planar homography. The two branches are independent—no stage in one depends on an output of the other until they converge—and this independence is exploited at runtime, since pitch keypoints change slowly relative to player positions and the right branch is therefore executed at a lower frequency than the left.

**Models and rule-based components.** The appearance stages are fully model-based. A generic YOLO detector is run on each frame and produces bounding boxes for players, goalkeepers, and referees; a second YOLO detector dedicated to the ball is run in parallel on the same frame and is the authoritative source of ball localisation

(the ball-class outputs of the generic detector are discarded, for reasons given in Section 3.6). Pitch landmarks are produced by a keypoint model. The team identification stage combines a pretrained vision-language embedding model (SigLIP) with dimensionality reduction (UMAP) and a small unsupervised clustering step (k-means,  $k = 2$ ). The tracker is a learning-free multi-object association algorithm (ByteTrack, with BoT-SORT selectable) applied to the generic detector’s player, goalkeeper, and referee outputs only; the ball is never passed to the tracker and instead passes through a lightweight temporal filter (nearest neighbour to the previous accepted position under a displacement gate) described in Section 3.6. All stages downstream of projection—heatmap accumulation, speed and distance computation, possession, pass detection, offside geometry—are rule-based: they operate deterministically on projected trajectories and carry no learned parameters.

**Caching and stride.** Two caching mechanisms are used to control the per-frame computational cost. First, pitch keypoint inference is run once every `pitch-stride` frames (default 15) and the resulting homography is reused on intervening frames; when a new inference fails to produce a valid estimate, the last valid homography is retained. Second, team classification is fitted once during a warmup window at the start of the video (default 20 s in the production preset) and reused thereafter; per-track team labels are produced by applying the fitted classifier to new crops and are stabilised by a majority-vote smoother over a 30-frame window. No other stage caches state across frames beyond the track state held internally by the tracker.

**Execution model.** The pipeline is driven by a single orchestration loop in `src/pipeline/run.py` that reads frames sequentially, invokes the stages in the order shown in Figure 3.1, and writes outputs to disk. The loop is deliberately sequential rather than concurrent: the stages have strict data dependencies, intermediate outputs are required for inspection and evaluation (NFR6), and real-time operation is explicitly out of scope (NFR7).

### 3.3.2 Data Flow and Intermediate Representations

The data objects passed between stages are summarised in Table 3.1. Where possible the pipeline reuses the `supervision` library’s native `Detections` type as the common currency of the detection–tracking–team-identification branch, which simplifies inter-operation with the off-the-shelf tracker and annotation utilities. The cost of this choice

| Object                      | Fields                             | Producer                      | Consumer(s)             |
|-----------------------------|------------------------------------|-------------------------------|-------------------------|
| Raw detection set           | xyxy,<br>confidence,<br>class_id   | Detector (§3.6)               | Tracker                 |
| Tracked detection set       | + tracker_id                       | Tracker (§3.6)                | Team ID, projection     |
| Team-labelled detection set | class_id over-written              | Team ID (§??)                 | Projection, analytics   |
| Ball detection (filtered)   | xyxy, confidence                   | Ball detector + filter (§3.6) | Projection, analytics   |
| Ball detections (generic)   | xyxy,<br>confidence,<br>tracker_id | Generic detector + tracker    | <i>Discarded</i>        |
| View transformer            | $H \in \mathbb{R}^{3 \times 3}$    | Pitch registration (§??)      | Projection              |
| Projected positions         | pitch coords (cm)                  | Projection (§??)              | Heatmaps, stats, events |
| Event record                | $(t, \tau, I)$                     | Event reasoning (§??)         | Logs, exports           |

Table 3.1: Data objects passed between pipeline stages. The `supervision.Detections` type is used throughout the left branch of the pipeline with fields progressively populated and, in the case of `class_id`, reinterpreted.

is that per-stage metadata that does not fit the `Detections` schema must be handled out of band, as described below.

**Detection and tracking record.** The core data object in the left branch is `supervision.Detections` with four populated fields. A raw detection set  $\mathcal{D}$  of  $N$  objects carries  $\mathbf{B} \in \mathbb{R}^{N \times 4}$  for bounding boxes in absolute pixel coordinates  $(x_1, y_1, x_2, y_2)$ ,  $\mathbf{c} \in [0, 1]^N$  for per-detection confidences, and  $\mathbf{k} \in \{0, 1, 2, 3\}^N$  for class identifiers under the detector’s label set  $\{0:\text{ball}, 1:\text{GK}, 2:\text{player}, 3:\text{referee}\}$ . The tracker additionally populates  $\mathbf{t} \in \mathbb{Z}^N$  with integer track identifiers on frames where association has succeeded; on frames where the tracker has produced no identifier, this field is absent.

**Encoding of team membership.** Team membership is encoded by overwriting the `class_id` field in place rather than by adding a new field to the detection record. Following team assignment, the semantics of `class_id` change: outfield players carry `class_id`  $\in \{0, 1\}$  denoting team A or team B, goalkeepers are resolved to the same two-valued alphabet where possible and to the value 2 otherwise, and referees are

forced to 2. This reinterpretation is deliberate: downstream renderers and exporters consume the field uniformly as a palette index, and the reuse avoids maintaining two parallel arrays that must be kept in lockstep. It is documented here explicitly because the value 2 denotes *referee or unresolved goalkeeper* after team assignment, not *player* as in the raw detector output.

**Ball as a distinct data stream.** The ball is handled as a separate `supervision.Detections` instance throughout the pipeline. It is never merged with the outfield detection set, never carries a `tracker_id`, and is consumed by downstream stages (bird’s-eye render, offside detector, stats tracker) as a parallel argument alongside the outfield detections. On frames where the temporal filter rejects all candidates, the ball set is empty; downstream consumers treat this condition as a missing-ball frame rather than as an error. The rationale for this parallel-stream design is given in Section 3.6.

**Pitch geometry and projected coordinates.** Pitch geometry is carried on a separate `ViewTransformer` object rather than attached to detections. The transformer encapsulates the  $3 \times 3$  homography  $H$  from broadcast-image pixels to a canonical pitch plane expressed in centimetres. Downstream consumers that require pitch-space positions (heatmap accumulation, speed and distance computation, possession, pass detection, offside geometry) receive the detection set and the transformer as separate arguments and compute  $\tilde{\mathbf{p}} = H \cdot [x, y, 1]^\top$  internally from the bottom-centre anchor of each bounding box, applying the standard perspective divide. This out-of-band handling mirrors the fact that geometry changes on a different schedule from appearance: a single transformer is used across all detections in a frame, and often across multiple frames under the keypoint-stride caching scheme.

**Event records.** Higher-level outputs produced by the analytics stage (possession segments, pass events, offside candidate frames) are represented as discrete records of the form  $(t, \tau, I)$ , where  $t$  is a frame timestamp,  $\tau$  is an event type, and  $I$  is a reference to the tracks and teams involved in the event. Records are accumulated across the video and exported as structured logs alongside the rendered visualisations.

**Persisted artefacts.** Each run produces a deterministic set of on-disk artefacts under a timestamped output directory: the annotated broadcast video, the bird’s-eye render (when enabled), the side-by-side composite (when enabled), per-track heatmap images, per-track movement summaries, a possession timeline, a pass event log, and a

flagged-frames list for offside candidates. These artefacts together form the input to the evaluation procedure described in Section 3.10 and are the basis of the results reported in Chapter 5.

### 3.3.3 System Outputs

The system produces three families of outputs, corresponding to the three principal ways the results are consumed: inspection of the broadcast through augmented video, compact summary through bird’s-eye visualisation, and analytical use through numerical exports.

**Augmented broadcast.** The annotated broadcast video renders per-frame detections, persistent track identifiers, and team colour coding back onto the source frame. This artefact is the most direct qualitative evidence that the pipeline is operating correctly: detection misses, identity switches, and team misassignments are immediately visible against the broadcast action.

**Bird’s-eye visualisation.** The bird’s-eye render projects every tracked outfield object, along with the ball, onto a scale drawing of the pitch. Objects are drawn as coloured discs whose colour encodes the (post-team-assignment) class identifier: two team colours for outfield players and resolved goalkeepers, a distinct colour for referees and unresolved goalkeepers, and a neutral colour for the ball. When an offside candidate has been flagged on a given frame, the offending player is recoloured and the relevant offside line is overlaid. The render is stateless: each frame is drawn from the current set of projected positions alone, without trails or accumulated history, which keeps the bird’s-eye faithful to the instantaneous pitch state. The bird’s-eye can be produced as a standalone video, as a component of a side-by-side composite with the broadcast, or both simultaneously.

**Tactical outputs and numerical exports.** The tactical analytics produce a set of quantitative outputs intended for post-hoc analysis rather than inline display. Per-player heatmaps capture occupation density in pitch space. Per-player movement summaries record distance covered and speed statistics over the sequence. A possession timeline records which team is in possession at each moment. Pass events and offside candidate frames are accumulated as event records. Each output is persisted in a structured, machine-readable form (CSV or JSON as appropriate to the shape of the data)

| Model            | Classes                           | Train | Val | Test |
|------------------|-----------------------------------|-------|-----|------|
| Generic detector | ball, goalkeeper, player, referee | 618   | 75  | 75   |
| Ball detector    | ball (single class)               | 932   | 114 | 119  |
| Pitch keypoints  | pitch (+ 32 keypoints)            | ~290  | 34  | 28   |

Table 3.2: Summary of the three training datasets. All datasets are hosted on Roboflow Universe and were downloaded via the Roboflow Python SDK; splits are those provided by the dataset version on Roboflow.

so that the evaluation procedure can compare against ground-truth annotations without re-running the pipeline, and so that the outputs can be consumed by downstream tools outside the scope of this project.

## 3.4 Datasets and Model Training

Three bespoke models are used in the pipeline: a generic object detector for outfield actors, a dedicated ball detector, and a pitch keypoint detector. Each was trained on a Roboflow-hosted dataset of broadcast football imagery, using the Ultralytics training harness with variants of the YOLOv8 architecture. This section describes the datasets (§3.4.1) and the training procedure for each model (§3.4.2, §3.5.4, §3.5.5). Results on held-out training splits are reported alongside the pipeline stage that consumes each model, where they inform the design discussion directly; full pipeline-level evaluation appears in Chapter 5. A fourth model used by the pipeline—the SigLIP vision-language encoder consumed by the team identification stage—is used off-the-shelf without fine-tuning and is described in Section ?? rather than here.

### 3.4.1 Datasets

All three training datasets were curated under a single Roboflow Universe workspace, were downloaded via the Roboflow Python SDK into the training environment, and were exported in YOLO format with the standard train/validation/test splits handled by Roboflow’s dataset versioning. Table 3.2 summarises the three datasets.

**Generic detector dataset.** The generic detector was trained on a Roboflow-hosted dataset of broadcast football frames curated collaboratively as a shared resource for broadcast football analysis, to which the author contributed additional annotations of frames sourced from a publicly available broadcast football dataset hosted on Kaggle.

The resulting dataset contains 768 annotated frames split 618/75/75 for training, validation, and test, and carries the four classes required by this work: *ball*, *goalkeeper*, *player*, and *referee*. The class distribution is heavily imbalanced: the *player* class accounts for the great majority of annotations, while goalkeepers, referees, and the ball each appear at most once per frame and sometimes not at all. This imbalance motivates the use of a separate, single-class dataset for the ball, described next.

**Ball detector dataset.** The dedicated ball detector was trained on a separate Roboflow-hosted dataset built specifically for small-object detection of the ball against broadcast backgrounds. It contains 1,165 annotated frames split 932/114/119 for training, validation, and test, with a single *ball* class; the Ultralytics log records the model’s class head being overridden from 80 to 1 on initialisation. Sourcing a ball-only dataset rather than extracting ball annotations from the generic dataset yields a substantially larger pool of positive examples (932 versus 618 frames) and permits augmentation, input resolution, and loss weighting to be tuned for a single small-object class without having to balance against the needs of the outfield classes.

**Pitch keypoint dataset.** The pitch keypoint model was trained on a Roboflow-hosted dataset of broadcast football frames annotated with 32 canonical pitch landmarks together with a pitch bounding box. The dataset is comparatively small (approximately 290 training frames, 34 validation, 28 test). Two factors make this size acceptable: keypoint annotation is considerably more labour-intensive than bounding-box annotation, and the geometric regularity of a football pitch—fixed markings at fixed distances—means that a relatively small dataset can cover the space of camera viewpoints encountered in broadcast footage. The keypoint layout defines a horizontal-flip symmetry (`flip_idx`) so that horizontal augmentation can be applied without destroying left-right keypoint identities.

### 3.4.2 Generic Object Detector Training

The generic detector was initialised from the pretrained `yolov8m.pt` checkpoint and fine-tuned for 50 epochs on the four-class dataset. Training was run on a Tesla T4 GPU in a Google Colab environment, at an input resolution of  $1280 \times 1280$  to preserve fine detail on small or distant players, with a batch size of 4 constrained by GPU memory at that resolution. The Ultralytics `auto` optimiser selected AdamW with default learning-rate schedules. Augmentation settings were the Ultralytics defaults for a detection



task (HSV jitter, mosaic, horizontal flip, random erasing, and RandAugment), with rotation, shear, and perspective augmentation disabled as inappropriate for a broadcast view with a largely fixed horizon.

**Artefact retention.** The final weights from this run were deployed to Roboflow for downstream use and retrieved from that deployment for pipeline inference. The `per-epoch results.csv` from the training run was not preserved in the project repository, so loss and mAP trajectories over training cannot be reconstructed. The final validation and test summaries emitted at the end of training are retained in the training notebook and are reported in §3.6, where they inform the design choice of treating the ball as a separate detection stream. This is a reproducibility limitation that the remaining two training runs, reported below, avoid.

## 3.5 Small-Object Detection and Dedicated Ball Detection

Although the ball is the primary object of interest in football, it is also the hardest to detect reliably in broadcast footage. At typical broadcast resolution, the ball subtends only a few dozen pixels in diameter, moves rapidly enough to produce motion blur, and is frequently occluded by players or absent from the frame entirely [13]. These properties place it firmly in the small-object detection regime, where standard detectors trained jointly on all classes tend to underperform because larger, more visually prominent objects dominate the training signal [42].

### 3.5.1 Why Small Objects Are Harder to Detect

The COCO benchmark defines a small object as one whose bounding box occupies fewer than  $32 \times 32$  pixels [23]. By this standard, the ball in a wide-angle broadcast frame is consistently small, often below  $20 \times 20$  pixels depending on camera zoom.

Small-object detection is inherently harder for two reasons. First, the IoU criterion (Equation (2.3)) becomes disproportionately sensitive to localisation error at small scales: a fixed pixel displacement  $\delta$  produces a much larger IoU drop on a small box than on a large one. As shown in Figure 3.2, a 2-pixel shift reduces IoU from 1.0 to 0.78 on a  $16 \times 16$  box but only to 0.94 on a  $64 \times 64$  box. This makes ball detection

acutely sensitive to blur, noise, and detector uncertainty. Second, in convolutional architectures the spatial resolution of feature maps decreases with depth; by the deepest layers, small objects may occupy only a single cell or be lost entirely. The FPN neck described in Section 2.4.3 partially mitigates this, but the representation of very small objects remains inherently weaker than that of larger ones [22].

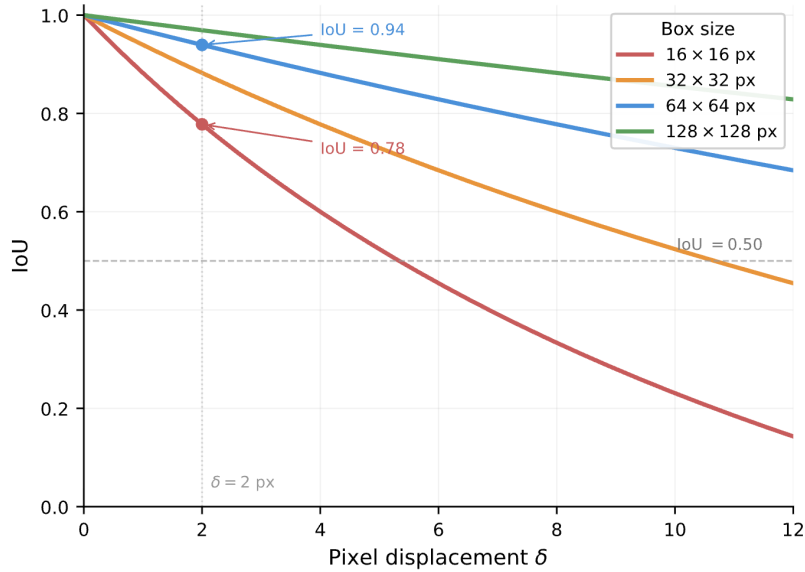


Figure 3.2: Effect of pixel displacement  $\delta$  on IoU for square bounding boxes of varying size. A single-axis shift is applied to the predicted box while the ground-truth box remains fixed. At  $\delta = 2$  pixels, a  $16 \times 16$  box drops to  $\text{IoU} = 0.78$  while a  $64 \times 64$  box retains  $\text{IoU} = 0.94$ , illustrating why small-object detection is disproportionately sensitive to localisation error. The dashed line marks the standard  $\text{mAP}_{50}$  threshold.

### 3.5.2 Strategies for Small-Object Detection

Several strategies have been proposed to address these difficulties. Multi-scale detection heads, as used in FPN-based architectures, are the most widely adopted architectural response; some methods extend this with additional high-resolution prediction scales or dilated convolutions [22]. At the data level, augmentation techniques such as copy-pasting small object instances into new training locations improve recall on under-represented size categories [18], while inference-time tiling increases effective resolution at the cost of higher computation.

A complementary approach is to train a dedicated detector whose capacity, input resolution, and confidence thresholds are tuned specifically for the small-object class.

This avoids the class-imbalance problem inherent in joint training and allows the operating point to be optimised independently. In particular, operating at a low confidence threshold  $\tau$  maximises recall at the cost of increased false positives — a trade-off that can be managed by downstream temporal filtering rather than within the detector itself [13].

### 3.5.3 Temporal Filtering of Ball Candidates

When a detector operates at a low confidence threshold, the resulting candidate set at each frame contains both the true ball and spurious detections. The general form of this set at frame  $t$  is

$$C_t = \left\{ (\mathbf{b}_t^{(j)}, s_t^{(j)}) \right\}_{j=1}^{N_t}, \quad (3.1)$$

where  $\mathbf{b}_t^{(j)}$  is a candidate location and  $s_t^{(j)}$  its confidence score.

Temporal coherence provides a principled basis for disambiguation. Because the ball follows a physically continuous trajectory, its inter-frame displacement is bounded even under rapid motion. Two families of filtering exploit this property. Motion-model approaches, such as the Kalman filter described in Section 2.5.1, predict the ball’s next position using an explicit dynamical model and select the closest candidate. These are effective when the assumed model holds, but can diverge under the abrupt accelerations from kicks, deflections, and rebounds that are routine in football. An alternative family enforces only that inter-frame displacement remains below a physically plausible bound, discarding candidates that would imply impossible ball speeds regardless of confidence. This sacrifices predictive power during smooth motion but is more robust to the sharp trajectory changes characteristic of ball dynamics.

The choice between these families, and the specific instantiation used in this project, is discussed in Chapter 3. Chapter 5 reports ball detection performance separately from the general object detector.

### 3.5.4 Dedicated Ball Detector Training

The dedicated ball detector was initialised from the pretrained `yolo8x.pt` checkpoint and fine-tuned for 150 epochs at input resolution  $1280 \times 1280$ , with a batch size of 8, early-stopping patience of 30 epochs, and the Ultralytics `auto` optimiser resolving to AdamW with a learning rate of  $2 \times 10^{-3}$ . Augmentation settings mirrored those used for the generic detector, with `mixup` and `copy_paste` disabled. Training was

carried out on an NVIDIA A100-80GB GPU on the University of Manchester CSF3 HPC facility, with mixed-precision (AMP) enabled throughout, and the full run took approximately 109 minutes of wall-clock time. The full training log and per-epoch `results.csv` are retained under `artifacts/ball_detection_model_yolov8-x/`, allowing the training curves to be reproduced exactly.

**Choice of the larger variant.** The ball detector uses the `yolov8x` variant (68.1 M parameters) rather than the `yolov8m` variant used for the generic detector. The motivation is that the ball is a small, fast-moving object whose appearance offers very little discriminative signal per pixel, and a higher-capacity backbone is better able to extract useful features from the few pixels on target. Because the dedicated detector runs only once per frame on the same input as the generic detector, and because the ball pipeline does not participate in per-frame tracking, the additional inference cost of the larger variant is acceptable within the overall runtime budget.

### 3.5.5 Pitch Keypoint Detector Training

The pitch keypoint model was initialised from the pretrained `yolov8x-pose.pt` checkpoint and fine-tuned for 500 epochs at input resolution  $640 \times 640$ , with a batch size of 48, early-stopping patience of 100 epochs, and the Ultralytics `auto` optimiser. Training was carried out on an NVIDIA A100-40GB GPU in a Google Colab environment, with mixed-precision enabled throughout, and the full run took approximately 45 minutes of wall-clock time. The full training log and per-epoch `results.csv` are retained under `artifacts/pitch_keypoint_detection_model/`.

**Mosaic disabled for keypoint training.** One departure from the default augmentation used elsewhere is that mosaic augmentation was explicitly disabled. Mosaic composes four images into a single training image by tiling; for bounding-box detection this is helpful (it increases the number of objects seen per batch and encourages context-invariance), but for keypoint tasks the tiling disrupts the geometric relationships between keypoints that the model must learn, since keypoints near the tile boundaries may be cropped or relocated in ways that do not correspond to any real camera viewpoint. Horizontal flip was retained because the keypoint layout defines an explicit left-right symmetry via `flip_idx`; vertical flip and perspective augmentation were disabled for the same geometric-integrity reasons.

**Deployment path.** The trained weights were exported and deployed to Roboflow for hosted inference, and the pipeline consumes the keypoint model through Roboflow’s hosted inference API rather than loading the local weights directly. The rationale for this deployment choice is explained in Section ??, which discusses the pitch registration stage that consumes this model’s output.

## 3.6 Detection and Tracking

The first operational stage of the pipeline detects the visual entities required for downstream football analysis and assigns persistent identities to players across frames. This stage is critical because later components, including team identification, pitch-space projection, possession estimation, heatmaps, and offside candidate detection, all depend on the reliability of the detections and tracks produced here.

### 3.6.1 Main Object Detection

The main detector is a YOLOv8m object detection model trained to recognise four classes: players, goalkeepers, referees, and the ball. During inference, each video frame is passed through the detector at an image size of 1280 pixels. The runtime confidence threshold is set to 0.2 and the IoU threshold is set to 0.5. These values were chosen to retain sufficient detections for the tracker while still removing low-confidence false positives.

The detector output for each frame consists of a set of bounding boxes, class labels, and confidence scores. These detections are converted into `supervision` detection objects so that they can be passed consistently between the object detection, tracking, annotation, and tactical analysis modules. Preserving this intermediate representation also makes it easier to inspect and evaluate the behaviour of individual pipeline stages.

Although the main detector includes a ball class, the ball is substantially harder to detect than players because it occupies very few pixels and is often blurred or occluded. For this reason, the main detector is used primarily for players, goalkeepers, and referees, while ball localisation is strengthened using a separate dedicated detector.

### 3.6.2 Dedicated Ball Detection and Filtering

A dedicated YOLOv8x ball detector is applied to each frame to improve the reliability of ball localisation. This design decision was made because errors in ball detection

propagate directly into possession, pass, and offside analysis. A missed or incorrect ball location can cause the system to assign possession to the wrong player, miss a pass transition, or trigger an incorrect offside candidate event.

The ball detector is run with a low initial confidence threshold (`ball_conf = 0.05`) so that weak ball candidates are not discarded too early. Candidate detections are then filtered temporally. A ball candidate is accepted only if it is sufficiently confident or spatially consistent with the previous accepted ball position. In particular, the pipeline uses a maximum jump threshold of 80 pixels and a minimum stronger-confidence threshold of 0.25. This prevents isolated low-confidence detections far from the previous ball position from being selected as the ball.

When multiple ball candidates are present, the system selects the candidate that is most consistent with the recent ball position. This filtering strategy does not solve all cases of ball occlusion or fast motion, but it reduces the effect of isolated false positives and produces a more stable ball trajectory for downstream tactical reasoning.

### 3.6.3 Multi-object Tracking

Player tracking is performed using tracking-by-detection. In each frame, the main YOLO detector supplies bounding boxes to a tracker, which assigns a persistent track identifier to each detected player. The default tracker used in the pipeline is ByteTrack, with BoT-SORT available as an alternative configuration. The tracker is selected through the runtime tracker configuration, allowing the same pipeline to be evaluated with different tracking algorithms.

ByteTrack was selected as the default tracker because it is designed to use both high-confidence and lower-confidence detections during association. This is useful in broadcast football footage, where players are frequently partially occluded and their detection confidence may temporarily fall even when they remain visible. By retaining lower-confidence detections for a second association step, ByteTrack can reduce unnecessary track fragmentation in crowded scenes.

The tracker returns detections with associated track identifiers. These track identifiers are preserved in the pipeline and are used by later stages to accumulate player positions, smooth team labels, generate heatmaps, and compute player-level statistics. Detections without valid track identifiers are not used for player-level temporal statistics, since they cannot be linked reliably across frames.

### **3.6.4 Track Handling and Failure Cases**

The tracking stage is vulnerable to several common failure cases. Occlusion can cause a player to disappear temporarily, leading to a broken track. Similar player appearance can cause identity switches, particularly when players cross or cluster around the ball. Camera motion can also affect the apparent movement of detections and make association more difficult.

The pipeline mitigates these issues in two ways. First, the tracker itself uses temporal association to preserve identities across short interruptions. Second, downstream stages avoid relying on a single frame wherever possible. For example, team labels are smoothed across a fixed track history, and movement statistics include a maximum-speed gate to reduce the effect of spurious projected movements.

Nevertheless, tracking errors cannot be fully eliminated at this stage. A fragmented or incorrect track can affect player heatmaps, distance estimates, team-level possession, and offside candidate detection. For this reason, the evaluation considers both the component-level behaviour of the tracker and the effect of tracking errors on the integrated pipeline.

## **3.7 Team Identification**

After player detections have been assigned track identifiers, the next stage is to infer the team membership of each tracked player. This is required for team-level outputs such as possession statistics, pass events, offside candidate detection, and team-coloured visualisations. The system performs team identification without match-specific manual labels by fitting an unsupervised classifier from player crops sampled from the input video.

### **3.7.1 Warm-up Sampling and Player Cropping**

Team identification begins with a warm-up phase at the start of each video. During this phase, frames are sampled from the first 20 seconds of footage using a stride of 30 frames. The main detector is applied to these sampled frames, and only detections classified as players are retained. Very small crops are discarded using a minimum bounding-box size of 35 by 35 pixels, since such detections usually contain too little visual information for reliable team assignment.

For each retained player detection, the system extracts a torso crop rather than

using the full bounding box. The crop is taken from approximately 15% to 65% of the bounding-box height. This focuses the representation on the shirt region, which usually contains the strongest team-specific visual signal, while reducing noise from shorts, legs, grass, and nearby players. A maximum of 800 warm-up crops is used, and at least 50 crops are required before the classifier is fitted.

### 3.7.2 Embedding and Clustering

The retained torso crops are used to fit a team classifier. The classifier extracts visual embeddings from each crop using a SigLIP-based image encoder, then reduces the embedding dimensionality using UMAP and clusters the reduced representations using K-means with  $k = 2$ . The two clusters correspond to the two outfield teams.

This approach was selected because it avoids requiring labelled team examples for every new match. A supervised team classifier would need new annotations whenever kits, lighting conditions, competitions, or broadcast styles changed. By fitting the classifier from the input video itself, the system can adapt to the visual appearance of the teams in the current match.

During the per-frame pipeline, each tracked player crop is passed through the same embedding and clustering pipeline to obtain a predicted team label. The output is a binary team assignment for outfield players. These assignments are then stabilised using temporal smoothing before being used by downstream analytics.

### 3.7.3 Temporal Smoothing and Role Handling

Frame-level team predictions can fluctuate because of occlusion, motion blur, partial crops, lighting variation, or players turning away from the camera. To reduce this instability, the system applies track-level majority-vote smoothing. For each track identifier, the most recent team predictions are stored in a fixed-length history of 30 predictions. The smoothed team label is then taken as the majority label within this history.

This design reduces the effect of isolated incorrect predictions while still allowing the team label to update if a track was initially misclassified. Smoothing is especially important because team labels are used in later components where a single-frame error can affect possession, pass, or offside logic.

Goalkeepers and referees require separate handling. Referees are assigned to a non-team class because they do not contribute to team possession or offside analysis. Goalkeepers are detected as a separate object class and are assigned to the team whose



outfield-player centroid is nearest in image space. This spatial rule is used because goalkeeper kits often differ from the outfield players' kits, making appearance-based clustering less reliable for this role.

The resulting representation assigns each detection either to team 0, team 1, or a non-team class. This representation is used consistently by the visualisation, possession, pass, and offside modules.

## **3.8 Pitch Registration**

Pitch registration maps image-space detections from the broadcast frame onto a canonical football pitch. This stage is required because image coordinates are affected by camera position, zoom, and perspective distortion. Tactical outputs such as heatmaps, speed, distance, possession proximity, and offside candidate detection therefore need to be computed in pitch space rather than directly in the broadcast image.

### **3.8.1 Pitch Keypoint Detection**

Pitch landmarks are detected using a YOLOv8x-pose model trained to localise 32 canonical pitch keypoints. For each processed frame, the model predicts a set of keypoint coordinates and confidence scores. Keypoints with confidence below 0.5 are discarded before homography estimation. This filtering step reduces the effect of uncertain keypoint predictions, which can otherwise produce unstable or incorrect pitch transformations.

A homography is estimated only when at least six valid keypoints are available. Although four point correspondences are theoretically sufficient to estimate a homography, requiring more than four points makes the transformation less dependent on any single detected landmark. Frames with too few reliable keypoints reuse the most recent valid transformation where possible, rather than forcing an unstable new estimate.

### **3.8.2 Homography Estimation and Coordinate Transformation**

For each valid set of pitch keypoints, the system estimates a homography between the detected image-space landmarks and their corresponding locations on a canonical pitch model. The homography provides a projective transformation from image coordinates to pitch coordinates. Once this transformation is available, detected object positions can be expressed in pitch space.

For players, goalkeepers, and referees, the image-space position is taken as the bottom-centre point of the bounding box, since this approximates the point where the player contacts the ground. For the ball, the centre of the bounding box is used. These points are then transformed into pitch coordinates using the estimated homography.

The pitch coordinate system is represented in centimetres, with a pitch length of 12,000 cm. Using metric-like pitch units allows later stages to express distance thresholds and spatial rules in interpretable terms. For example, the possession and offside modules use distances in centimetres rather than pixel distances, making the thresholds less dependent on camera zoom.

### **3.8.3 Caching and Robustness Measures**

Estimating a new pitch transformation on every frame is computationally expensive and can also introduce visual jitter when keypoint predictions vary slightly between consecutive frames. To reduce this problem, the pipeline recomputes the pitch transformation every 15 frames and caches the most recent valid transformation between updates.

This stride-cached design is a compromise between accuracy, stability, and runtime. Updating the homography too rarely can make the transformation lag behind camera motion, while updating it every frame can amplify keypoint noise. A stride of 15 frames provides regular updates while reducing unnecessary recomputation.

Pitch registration remains one of the most important sources of downstream error. If the homography is inaccurate, then all pitch-space outputs derived from it may be affected, including heatmaps, speed and distance estimates, possession proximity, and offside geometry. For this reason, registration quality is inspected both visually through bird’s-eye outputs and quantitatively where keypoint validation metrics are available.

## **3.9 Tactical Output Generation**

Once detections have been tracked, assigned team labels, and transformed into pitch coordinates, the system generates tactical outputs from the resulting structured representation. These outputs are intended to make the broadcast video more analytically useful by converting low-level detections into visual summaries, player statistics, team possession estimates, pass events, and candidate offside situations.

### 3.9.1 Bird's-eye Visualisation

The bird's-eye visualisation represents the detected players, goalkeepers, referees, and ball on a canonical top-down pitch. For each frame with a valid pitch transformation, the image-space object positions are projected into pitch coordinates and rendered on the pitch model. Players are coloured according to their inferred team labels, referees are shown as a separate non-team class, and the ball is rendered using its filtered pitch-space position when available.

This visualisation provides an interpretable way to inspect whether the detection, team assignment, and pitch registration stages are behaving coherently. It is also used to generate side-by-side videos, where the original broadcast frame and the corresponding pitch-space representation are displayed together. This makes it easier to identify errors such as incorrect team labels, unstable homographies, or ball locations that do not match the broadcast footage.

### 3.9.2 Heatmaps, Speed and Distance

Player heatmaps are generated by accumulating each tracked player's pitch-space positions over time. The pitch is divided into a fixed grid of  $480 \times 280$  cells, and each valid player position contributes to the corresponding grid location. Heatmaps are generated only when at least 40 valid samples are available for a track, reducing the likelihood of producing misleading visualisations from short or fragmented tracks.

To improve readability, the accumulated heatmap is smoothed using a Gaussian blur with kernel size 31 and a gamma correction of 0.22. The final heatmap is saved both as an image and as a raw numerical array, so that the visual output can be inspected while the underlying positional distribution remains available for further analysis.

Speed and distance estimates are computed from changes in player pitch-space position over time. Consecutive valid positions for each track are used to estimate displacement, which is accumulated to produce total distance travelled. Speed is derived from displacement and frame timing. Since small errors in pitch registration can produce unrealistically large apparent jumps, the system applies a maximum-speed gate of 40 km/h. Movements above this threshold are treated as noise and excluded from the accumulated statistics.

### 3.9.3 Possession and Pass Detection

Possession is estimated using the pitch-space distance between the filtered ball position and nearby players. For each frame, the system identifies the player closest to the ball. A player is considered a possession candidate only if the distance to the ball is below 350 cm. To reduce noise from brief misdetections or transient ball movements, possession must be stable for at least three consecutive frames before being confirmed.

Team possession is then obtained from the confirmed player’s team label. If no player satisfies the proximity condition, the frame is treated as having no new confirmed possession event. This conservative design reduces the risk of assigning possession during periods when the ball is airborne, occluded, or incorrectly detected.

Pass events are detected from stable changes in possession. A pass is recorded when confirmed possession changes from one player to another in a temporally stable way. This rule-based definition is intentionally simple: it is designed to identify likely possession transfers rather than to provide a complete semantic classification of football actions. As a result, the pass output is best interpreted as a set of candidate pass events that require evaluation against manually inspected clips.

### 3.9.4 Offside Candidate Detection

Offside candidate detection is performed in pitch space using the detected players, inferred team labels, ball position, and estimated attacking direction. The attacking direction is inferred automatically from accumulated team  $x$ -centroids once at least 30 valid team-position samples are available, rather than being manually specified for each video.

For each frame with sufficient information, the system estimates the offside line from the second-last defender’s pitch-space position. If only one defender is available, a fallback line is computed using the available defensive player, although this case is less reliable. An attacker is treated as a candidate offside player if they are beyond the defensive line by more than a 30 cm margin at the relevant moment and are spatially close enough to the ball event context to be considered involved in play.

The output of this module is not intended to replicate full official offside judgement. The official law depends on contextual factors such as the exact moment the ball is played, active involvement, deliberate play by defenders, and body-part eligibility. The system instead detects candidate offside situations under simplified geometric

assumptions. Candidate events are saved as structured JSON records and visualised using an overlay for a fixed display duration of 50 frames, allowing them to be inspected during evaluation.

## **3.10 Evaluation Methodology**

The evaluation was designed to assess both the individual components of the pipeline and the behaviour of the integrated system. This distinction is important because strong component-level performance does not necessarily imply that the final tactical outputs are reliable. For example, a detector may achieve high validation accuracy while still missing the ball in key moments, or a pitch registration model may perform well on isolated frames while producing unstable transformations during camera motion. The evaluation therefore considers not only model-level metrics, but also the extent to which the complete pipeline produces coherent and useful football analysis outputs.

### **3.10.1 Component-level Evaluation**

The first level of evaluation assesses the supervised models used by the pipeline. The main object detector, dedicated ball detector, and pitch keypoint detector are evaluated using their validation or test splits, with metrics appropriate to each task.

For the object detection models, performance is measured using precision, recall, mAP@50, and mAP@50–95. These metrics assess both whether the relevant objects are detected and how accurately their bounding boxes are localised. The dedicated ball detector is evaluated separately because ball localisation is a known bottleneck for downstream possession, pass, and offside analysis.

For the pitch keypoint model, evaluation considers both bounding-box and keypoint performance where available. The keypoint metrics are particularly important because pitch registration depends on the geometric accuracy of the detected landmarks rather than only on whether the pitch itself is detected.

Tracking is evaluated by comparing the stability and continuity of player tracks across representative clips. Where manual annotations are available, track fragmentation, identity switches, and track continuity are assessed directly. Where dense tracking ground truth is not available, the tracker is evaluated through visual inspection, track-count consistency, and its effect on downstream outputs such as heatmaps and distance estimates.

### 3.10.2 End-to-end Pipeline Evaluation

The second level of evaluation assesses the complete pipeline on broadcast football clips. This evaluation focuses on whether the system produces coherent outputs after all components have been combined. The main outputs considered are annotated broadcast videos, bird’s-eye visualisations, side-by-side videos, heatmaps, statistics files, and offside event records.

End-to-end evaluation is performed using representative clips containing common broadcast conditions, including camera panning, zooming, player occlusion, ball movement, and changes in pitch visibility. The outputs are inspected to assess whether detected entities remain visually consistent, whether team labels are stable, whether pitch-space projections remain aligned with the broadcast view, and whether the generated statistics are plausible.

Where manual labels are produced, tactical outputs are evaluated against manually inspected ground truth. Possession estimates are compared with the observed player or team in control of the ball, pass events are compared with manually identified possession transfers, and offside candidates are compared with visible attacking situations in the clip. This provides a more meaningful assessment of tactical usefulness than model-level metrics alone.

### 3.10.3 Ablation Studies

Ablation studies are used to evaluate the contribution of key design decisions. Each ablation changes one component or configuration while keeping the rest of the pipeline fixed, allowing the effect of that design choice to be isolated.

The planned ablations are:

- **Dedicated ball detector.** The pipeline is compared with and without the dedicated ball detector to determine whether the specialised model improves possession, pass, and offside outputs.
- **Tracker choice.** ByteTrack and BoT-SORT are compared on the same clips to assess differences in track continuity, identity stability, and downstream statistics.
- **Team-label smoothing.** Raw frame-level team predictions are compared with track-level majority-vote smoothing to assess whether smoothing reduces label instability.

- **Pitch registration stride.** Different homography update intervals are compared to measure the trade-off between runtime, projection stability, and responsiveness to camera motion.
- **Speed filtering.** Distance and speed estimates are compared with and without the maximum-speed gate to determine how much unrealistic movement is introduced by registration noise.

These ablations are included because the project contains several design choices that could otherwise appear arbitrary. Comparing alternatives makes it possible to justify the final configuration and identify which components have the greatest effect on end-to-end reliability.

### 3.10.4 Runtime Evaluation

Runtime evaluation measures the practical cost of processing broadcast video. The system records total processing time and, where possible, the time spent in major pipeline stages such as detection, ball detection, pitch registration, team assignment, rendering, and tactical output generation.

Runtime is reported as processing time per frame and effective frames per second. Additional comparisons are made for configurations that change runtime substantially, such as different pitch registration strides or optional bird’s-eye and side-by-side rendering. This allows the evaluation to distinguish between methods that improve output quality and methods that are too expensive for practical use.

The aim is not necessarily to achieve real-time performance, since the project prioritises analytical quality and reproducibility. However, runtime remains an important non-functional requirement because a system that is too slow to run on complete clips would have limited practical value.

### 3.10.5 Error Analysis

Error analysis is used to examine where the pipeline fails and how errors propagate between stages. This is necessary because the final tactical outputs depend on several earlier components. A missed ball detection can affect possession and pass detection; an identity switch can distort heatmaps and distance estimates; an incorrect team label can affect possession and offside logic; and an unstable homography can corrupt all pitch-space measurements.

The evaluation therefore records representative failure cases and groups them by cause. The main categories considered are missed or false detections, ball tracking errors, track fragmentation, team-label switches, pitch registration failures, and tactical inference errors. For each category, the analysis identifies the likely cause, the downstream effect, and any mitigation used in the pipeline.

This error-focused evaluation is important because the project is an integrated system rather than a single model. The objective is not only to report whether the outputs are correct, but also to explain why they succeed or fail under different broadcast conditions. This provides the basis for the critical evaluation in Chapter 5.

## 3.11 Implementation and Reproducibility

The pipeline was implemented in Python, using a modular code structure so that individual stages could be configured, tested, and replaced independently. The main entry point is `src/track_video_supervision.py`, which parses the runtime configuration and calls the pipeline orchestration code in `src/pipeline/run.py`. This organisation separates high-level execution from the component-specific modules used for detection, team identification, pitch registration, visualisation, statistics, and offside detection.

### 3.11.1 Software Environment

The implementation uses Python 3.11. The main libraries are Ultralytics for YOLO model training and inference, Supervision for detection handling and video annotation utilities, OpenCV for image processing and video writing, NumPy for numerical operations, scikit-learn and UMAP for clustering-related operations, and Hugging Face Transformers for the SigLIP-based team embedding model. The Roboflow inference tools are used for the pitch model integration, and the Roboflow sports library is used for the team classification pipeline.

The project dependencies are recorded in `requirements.txt`. In addition, the Roboflow sports library is installed as an editable dependency from its source repository. The environment also requires a Roboflow API key and field model identifier, which are specified through environment variables. An example environment file is provided to document the required variables without exposing private credentials.



### 3.11.2 Configuration and Model Weights

The pipeline is controlled through runtime parameters rather than hard-coded settings. Important configuration values include the detector image size, detection confidence threshold, IoU threshold, tracker type, pitch registration stride, keypoint confidence threshold, ball filtering thresholds, team smoothing window, heatmap settings, and offside detection parameters.

The default inference configuration uses an image size of 1280 pixels, detector confidence threshold of 0.2, IoU threshold of 0.5, and ByteTrack as the tracker. The team classifier samples the first 20 seconds of video with a stride of 30 frames and applies a 30-frame smoothing window to team predictions. Pitch registration is updated every 15 frames, using keypoints with confidence at least 0.5 and requiring at least six valid keypoints before estimating a homography.

The trained model weights are stored separately from the source code. The required weights are the main object detector `models/object_detection/best.pt`, the dedicated ball detector `models/ball_detection/best.pt`, and the pitch keypoint detector `models/pitch_keypoints/best.pt`. To reproduce the full system, these weights must either be supplied with the project artefacts or regenerated using the documented training scripts and notebooks.

### 3.11.3 Running the Pipeline

The pipeline can be executed from the command line using the provided run scripts. The standard execution path processes an input video, writes an annotated output video, and optionally generates bird's-eye videos, side-by-side videos, heatmaps, statistics files, pitch debug videos, and offside event records. The main generated outputs are written to the `outputs/` directory.

For each input video, the core output is an annotated video showing detections, track identifiers, team labels, the ball marker, and offside overlays where enabled. Optional outputs include a bird's-eye video, a side-by-side broadcast-and-pitch view, JSON statistics, JSON offside-event records, and per-player heatmap files. The statistics output records team possession, passes, player speed estimates, and distance travelled, while the heatmap directory stores both image files and raw numerical arrays.

This output structure was chosen to support both visual inspection and later evaluation. Videos make it possible to inspect whether the integrated pipeline is behaving

plausibly, while JSON and numerical outputs allow quantitative comparison with manual labels and ablation results.

### **3.11.4 Reproducibility Limitations**

Several limitations affect exact reproducibility. First, the trained model weights are external artefacts and are not stored directly inside the source code repository. Reproducing the system therefore requires either access to the trained weights or re-training the models from the documented datasets and training configuration.

Second, some datasets and hosted models are accessed through Roboflow services, so reproduction depends on access to the relevant workspaces, dataset versions, and API credentials. Third, although YOLO training used deterministic settings and a fixed seed where supported, the runtime team classifier is fitted from sampled video crops and may vary if the underlying embedding, UMAP, or clustering behaviour is not fully deterministic.

Finally, the current implementation is primarily an experimental research pipeline rather than a fully packaged software product. It provides scripts, configuration parameters, model artefact locations, and documented outputs, but does not include a complete automated unit-test suite. This limitation is addressed in the evaluation through component-level validation, visual inspection, ablation studies, and end-to-end testing on representative clips.

## **3.12 Chapter Summary**

This chapter described the methodology used to construct the proposed broadcast football video analysis pipeline. The system was designed around a set of functional and non-functional requirements, with the central objective of producing tactical outputs from a single broadcast video without relying on commercial tracking infrastructure.

The methodology used a modular architecture in which object detection, ball detection, multi-object tracking, team identification, pitch registration, and tactical output generation are implemented as separable stages. This structure allows each component to be configured and evaluated independently, while still supporting end-to-end analysis of the complete pipeline.

The chapter also described the datasets, model training procedures, runtime configuration, and reproducibility measures used by the system. Particular attention was

given to design decisions that affect downstream reliability, including the use of a dedicated ball detector, track-level team-label smoothing, stride-cached pitch registration, and pitch-space rules for possession, pass, and offside candidate detection.

The following chapter reports the results produced by this methodology, including model-level validation results, pipeline outputs, tactical statistics, visualisations, and runtime measurements. Chapter 5 then evaluates these results against the requirements and success criteria defined in this chapter.

# **Chapter 4**

## **Results**

### **4.1 Experimental Setup**

### **4.2 Findings**

# **Chapter 5**

## **Evaluation**

### **5.1 Analysis**

### **5.2 Discussion**

### **5.3 Limitations**

# **Chapter 6**

## **Conclusion**

### **6.1 Summary**

### **6.2 Future Work**

# Bibliography

- [1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *International Conference on Database Theory (ICDT)*, pages 420–434. Springer, 2001.
- [2] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.
- [3] M. Beetz, S. Gedikli, J. Bandouch, B. Kirchlechner, N. von Hoyningen-Huene, and A. Perzylo. Visually tracking football games based on tv broadcasts. In *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.
- [4] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.
- [5] A. Cioppa, S. Giancola, A. Delière, L. Kang, X. Zhou, Z. Cheng, B. Ghanem, and M. Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3491–3502, June 2022.
- [6] J. Davis, L. Bransen, L. Devos, A. Jaspers, W. Meert, P. Robberechts, J. Van Haaren, and M. Van Roy. Methodology and evaluation in sports analytics: challenges, approaches, and lessons learned. *Machine Learning*, 113:6977–7010, 2024.
- [7] A. Delière, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasber, B. Ghanem, and M. Van Droogenbroeck. SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos. In *Proceedings of*

*the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 4508–4519, 2021.

- [8] P. Dendorfer, A. Osep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixe. Motchallenge: A benchmark for single-camera multiple target tracking. *International Journal of Computer Vision*, 129(4):845–881, 2021.
- [9] S. Ellens, D. Hodges, S. McCullagh, J. J. Malone, and M. C. Varley. Interchangeability of player movement variables from different athlete tracking systems in professional soccer. *Science and Medicine in Football*, 6(1):1–6, 2022.
- [10] J. Fernandez and L. Bornn. Wide open spaces: A statistical technique for measuring space creation in professional soccer. In *MIT Sloan Sports Analytics Conference*, 2018.
- [11] F. R. Goes, L. A. Meerhoff, M. J. O. Bueno, D. M. Rodrigues, F. A. Moura, M. S. Brink, M. T. Elferink-Gemser, A. J. Knobbe, S. A. Cunha, R. S. Torres, and K. A. P. M. Lemmink. Unlocking the potential of big data to support tactical performance analysis in professional soccer: A systematic review. *European Journal of Sport Science*, 21(4):481–496, 2021.
- [12] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [13] A. Gualtieri, E. Rampinini, A. Dello Iacono, and M. Beato. High-speed running and sprinting in professional adult soccer: Current thresholds definition, match demands and training strategies. a systematic review. *Frontiers in Sports and Active Living*, 5:1116293, 2023.
- [14] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2003.
- [15] M. Istasse, J. Moreau, and C. De Vleeschouwer. Associating players to teams using visual features trained with weak supervision. In *International Conference on Machine Vision Applications (MVA)*, 2019.
- [16] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8, 2023.
- [17] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.



- [18] M. Kisantal, Z. Wojna, J. Muber, A. Pirinen, and E. Rahtu. Augmentation for small object detection. In *9th International Conference on Advanced Concepts for Intelligent Vision Systems (ACIVS)*, 2019. arXiv preprint arXiv:1902.07296.
- [19] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [20] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [21] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [22] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.
- [23] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [24] D. Linke, D. Link, and M. Lames. Football-specific validity of TRACAB’s optical video tracking systems. *PLOS ONE*, 15(3):e0230179, 2020.
- [25] J. Liu, X. Tong, W. Li, T. Wang, Y. Zhang, and H. Wang. Automatic player detection, labeling and tracking in broadcast soccer video. *Pattern Recognition Letters*, 30(2):103–113, 2009.
- [26] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37. Springer, 2016.
- [27] L. Lolli, P. Bauer, C. Irving, D. Bonanno, O. Höner, W. Gregson, and V. Di Salvo. Data analytics in the football industry: a survey investigating operational frameworks and practices in professional clubs and national federations from around the world. *Science and Medicine in Football*, 2024.
- [28] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021.

- [29] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.
- [30] F. Magera, T. Hoyoux, O. Barnich, and M. Van Droogenbroeck. BroadTrack: Broadcast camera tracking for soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Tucson, Arizona, USA, February 2025.
- [31] A. Maglo, A. Orcesi, J. Denize, and Q. C. Pham. Individual locating of soccer players from a single moving view. *Sensors*, 23(18):7938, 2023.
- [32] L. McInnes, J. Healy, N. Saul, and L. Großberger. Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018.
- [33] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [34] K. O’Shea and R. Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [35] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.
- [36] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [37] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [38] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

- [39] V. Somers, V. Joos, A. Cioppa, S. Giancola, S. A. Ghasemzadeh, F. Magera, B. Standaert, A. M. Mansourian, X. Zhou, S. Kasaei, B. Ghanem, A. Alahi, M. Van Droogenbroeck, and C. De Vleeschouwer. Soccernet game state reconstruction: End-to-end athlete tracking and identification on a minimap. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3293–3305, June 2024.
- [40] The International Football Association Board. Laws of the game 2025/26. <https://downloads.theifab.com/downloads/laws-of-the-game-2025-26-single-pages?l=en>, 2025. Accessed 2026-04-08.
- [41] J. Theiner and R. Ewerth. Tvcilib: Camera calibration for sports field registration in soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1166–1175, 2023.
- [42] J. Theiner, W. Gritz, E. Müller-Budack, R. Rein, D. Memmert, and R. Ewerth. Extraction of positional player data from broadcast soccer videos. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1463–1473, 2022.
- [43] F. Vidal-Codina, N. Evans, B. El Fakir, and J. Billingham. Automatic event detection in football using tracking data. *Sports Engineering*, 25(1), 2022.
- [44] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [45] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.
- [46] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022*, volume 13682 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2022.

## **Appendix A**

### **Additional Materials**